



IMF Smart Education

Escuela de IA & Big Data

**Máster en Data Science y Business
Analytics**

**Impacto del uso problemático del
Internet en los niños y adolescentes**

TFM elaborado por: Anthony Manzueta Difo

Tutor de TFM : Juan Manuel Moreno Lamparero

Contenido

Resumen	1
Introducción y Antecedentes	2
Objetivos	3
Materiales y Métodos.....	4
Materiales	4
Base de datos:	4
Entorno de desarrollo:	9
Métodos	11
Entendimiento del negocio o del tema del proyecto:.....	11
Entendimiento de los datos:	11
Preprocesamiento de los datos:.....	11
Análisis exploratorio de los datos:.....	12
Modelización de los datos:	12
Validación de los modelos:	12
Resultado	13
Librerías.....	13
Funciones propias	13
Entendimiento y procesamiento de datos: dataset series	16
Distribuciones variables y Outliers	24
Clústeres: valores atípicos y valores nulos.....	29
Análisis Exploratorio de los datos	34
Análisis Univariante	34
Análisis Bivariante.....	45
Codificaciones variables categóricas	53
Análisis de correlación.....	53
Estandarización variables	57
Importancias variables RandomForestRegressor	57
PCA.....	60
Modelados datos	61
División del conjunto de datos en pruebas y entrenamiento	61
Modelo de regresión logística	61
Modelo de redes neuronales.....	62
Modelo XGBoots XGBClassifier	62
Evaluación y Validación: RandomForestClassifier	63
Evaluación y Validación: XGBootClassifier.....	65

Conclusiones	71
Referencia bibliográfica	72

Resumen

El uso problemático de internet en niños y adolescentes refleja una preocupación creciente, especialmente por su relación con problemas de salud mental como la depresión y la ansiedad. Sin embargo, los métodos actuales para su detección suelen requerir evaluaciones profesionales, lo que limita su accesibilidad y aplicabilidad en diversos contextos sociales y culturales.

Ante estas dificultades, se ofrece una opción que podrían cambiar las cosas: utilizar indicadores de condición física como niveles de actividad, postura y hábitos alimenticio, como señales indirectas del uso abusivo de tecnología. Estas medidas son accesibles, no invasivas y no requieren intervención clínica especializada.

Este enfoque plantea el desarrollo de modelos predictivos capaces de identificar patrones asociados al uso problemático de internet a partir de datos físicos infantiles, facilitando intervenciones tempranas. Con ello, se busca contribuir a la promoción de hábitos digitales saludables y a la mejora del bienestar integral de niños y adolescentes en entornos digitales.

Introducción y Antecedentes

En la actualidad, el acceso a internet y el uso intensivo de tecnologías digitales forman parte esencial de la vida cotidiana, especialmente entre niños y adolescentes. Si bien estas herramientas ofrecen beneficios significativos en términos educativos y recreativos, su uso excesivo o inadecuado puede conllevar efectos negativos sobre el bienestar físico, emocional y social de los jóvenes. Este hecho ha sido reconocido como "uso problemático del internet" (UPI), una condición que se ha relacionado con síntomas de ansiedad, depresión, aislamiento social, alteraciones del sueño y bajo rendimiento académico.

No obstante, identificar el UPI en niños y adolescentes representa un desafío importante. Las metodologías actuales suelen requerir evaluaciones clínicas especializadas o autoinformes validados, que no siempre están disponibles en todos los contextos sociales, culturales o económicos. Esta abertura pone límites a la capacidad de intervención temprana y perpetúa la exposición prolongada a factores de riesgo digital.

Frente a esta problemática, surge la necesidad de encontrar estrategias indirectas, accesibles y eficaces para la detección temprana del uso problemático del internet. Una vía prometedora es el análisis de variables relacionadas con la condición física infantil, como los patrones de sueño, alimentación, actividad física o postura corporal, los cuales han mostrado alteraciones frecuentes en usuarios con conductas digitales compulsivas. El uso de estos indicadores abre la puerta al desarrollo de modelos predictivos más inclusivos, con base en datos disponibles y sin la necesidad de herramientas clínicas complejas.

El UPI se ha definido como el uso compulsivo, descontrolado o interferente de la red, afectando el desarrollo funcional del individuo (Kuss & Griffiths, 2015).

Las herramientas tradicionales para identificar el UPI incluyen cuestionarios validados como el Internet Addiction Test (IAT) o entrevistas clínicas. Sin embargo, estas requieren personal capacitado, tiempo de aplicación y, en muchos casos, traducciones y adaptaciones culturales (Pontes et al., 2014), lo cual limita su uso masivo o en comunidades de bajos recursos.

Objetivos

Aplicar los conocimientos adquiridos en el máster de los diferentes modelos y técnicas de aprendizaje automático en este importante e interesante tema que afecta a los niños y adolescentes y a la vez desarrollar un modelo predictivo que, a partir de datos relacionados con la condición física de niños y adolescentes, permita identificar de forma temprana posibles casos de uso problemático del internet.

Materiales y Métodos

A continuación, los materiales utilizados para este proyecto:

Materiales

Base de datos:

Los datos para este proyecto los conseguí de la competencia de Kaggle que realizó Child Mind Institut sobre el uso problemáticos del internet. Este dataset , en general, estaba compuesto por dos tablas principales, una contenía las mediciones de los instrumentos que se utilizaron en las investigaciones, llamados train.csv y test.csv. Estos se describen a continuación por tipo instrumentos, nombres de campos y descripción, en el mismo orden:

Instrumento	Campo	Descripción
Identificador	ID	ID del participante
Demográfico	Basic_Demos-Enroll_Season	Temporada inscripción
Demográfico	Basic_Demos-Age	Edad del participante
Demográfico	Basic_Demos-Sex	Sexo del participante
Escala Evaluación Global Infantil	CGAS-Season	Temporada participación
Escala Evaluación Global Infantil	CGAS-CGAS_Score	Puntuación de la Escala Evaluación Global Infantil
Medidas físicas	Physical-Season	Temporada participación
Medidas físicas	Physical-BMI	Índice masa corporal (kg/m ²)
Medidas físicas	Physical-Height	Estatura (pulgadas)
Medidas físicas	Physical-Weight	Peso (libras)
Medidas físicas	Physical_Waist_Circumference	Perímetro cintura (pulgadas)
Medidas físicas	Physical-Diastolic_BP	Presión arterial diastólica (mmHg)
Medidas físicas	Physical-HeartRate	Frecuencia cardíaca (latidos/min)
Medidas físicas	Physical-Systolic_BP	Presión arterial sistólica (mmHg)
FitnessGram Vitals y cinta de correr	Fitness_Endurance-Season	Temporada de participación
FitnessGram Vitals y cinta de correr	Fitness_Endurance-Max_Stage	Etapas máxima alcanzada
FitnessGram Vitals y cinta de correr	Fitness_Endurance-Time_Mins	Tiempo exacto completado: Minutos
FitnessGram Vitals y cinta de correr	Fitness_Endurance-Time_Sec	Tiempo exacto completado: Segundos
FitnessGram Niño	FGC-Season	Temporada de participación
FitnessGram Niño	FGC-FGC_CU	Total de abdominales
FitnessGram Niño	FGC-FGC_CU_Zone	Zona de fitness de abdominales
FitnessGram Niño	FGC-FGC_GSND	Total de fuerza de agarre (no dominante)
FitnessGram Niño	FGC-FGC_GSND_Zone	Zona de fitness de fuerza de agarre (no dominante)
FitnessGram Niño	FGC-FGC_GSD	Total de fuerza de agarre (dominante)
FitnessGram Niño	FGC-FGC_GSD_Zone	Zona de fitness de fuerza de agarre (dominante)
FitnessGram Niño	FGC-FGC_PU	Total de flexiones

FitnessGram Niño	FGC-FGC_PU_Zone	Zona de fitness de flexiones
FitnessGram Niño	FGC-FGC_SRL	Total de sentadillas y estiramientos (lado izquierdo)
FitnessGram Niño	FGC-FGC_SRL_Zone	Zona de fitness de sentadillas y estiramientos (lado izquierdo)
FitnessGram Niño	FGC-FGC_SRR	Sentarse y alcanzar total (lado derecho)
FitnessGram para niños	FGC-FGC_SRR_Zone	Zona de ejercicios de Sit & Reach (lado derecho)
FitnessGram para niños	FGC-FGC_TL	Total de levantamiento de tronco
FitnessGram para niños	FGC-FGC_TL_Zone	Zona de ejercicios de levantamiento de tronco
Análisis de impedancia bioeléctrica	BIA-Season	Temporada de participación
Análisis de impedancia bioeléctrica	BIA-BIA_Activity_Level_num	Nivel de actividad
Análisis de impedancia bioeléctrica	BIA-BIA_BMC	Contenido mineral óseo
Análisis de impedancia bioeléctrica	BIA-BIA_BMI	Índice de masa corporal
Análisis de impedancia bioeléctrica	BIA-BIA_BMR	Tasa metabólica basal
Análisis de impedancia bioeléctrica	BIA-BIA_DEE	Gasto energético diario
Análisis de impedancia bioeléctrica	BIA-BIA_ECW	Agua extracelular
Análisis de impedancia bioeléctrica	BIA-BIA_FFM	Masa libre de grasa
Análisis de impedancia bioeléctrica	BIA-BIA_FFMI	Índice de masa libre de grasa
Análisis de impedancia bioeléctrica	BIA-BIA_FMI	Índice de masa grasa
Análisis de impedancia bioeléctrica	BIA-BIA_Fat	Porcentaje de grasa corporal
Análisis de impedancia bioeléctrica	BIA-BIA_Frame_num	Constitución corporal
Análisis de impedancia bioeléctrica	BIA-BIA_ICW	Agua intracelular
Análisis de impedancia bioeléctrica	BIA-BIA_LDM	Masa seca magra
Análisis de impedancia bioeléctrica	BIA-BIA_LST	Tejido blando magro
Análisis de impedancia bioeléctrica	BIA-BIA_SMM	Masa muscular esquelética
Análisis de impedancia bioeléctrica	BIA-BIA_TBW	Agua corporal total
Cuestionario de Actividad Física (Adolescentes)	PAQ_A-Season	Temporada de participación
Cuestionario de Actividad Física (Adolescentes)	PAQ_A-PAQ_A_Total	Puntuación del resumen de la actividad (adolescentes)
Cuestionario de Actividad Física (Niños)	PAQ_C-Season	Temporada de participación
Cuestionario de Actividad Física (Niños)	PAQ_C-PAQ_C_Total	Puntuación del resumen de la actividad (niños)
Test de adicción a internet entre padres e hijos	PCIAT-Season	Temporada de participación

Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_01	¿Con qué frecuencia desobedece su hijo los límites de tiempo que usted establece para usar internet?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_02	¿Con qué frecuencia descuida su hijo las tareas del hogar para pasar más tiempo en línea?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_03	¿Con qué frecuencia prefiere su hijo pasar tiempo en línea en lugar de con el resto de la familia?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_04	¿Con qué frecuencia entabla nuevas relaciones con otros usuarios de internet?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_05	¿Con qué frecuencia se queja del tiempo que su hijo pasa en línea?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_06	¿Con qué frecuencia se ven afectadas las calificaciones de su hijo por el tiempo que pasa en línea?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_07	¿Con qué frecuencia revisa su hijo su correo electrónico antes de hacer otra cosa?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_08	¿Con qué frecuencia parece su hijo aislarse de los demás desde que descubrió internet?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_09	¿Con qué frecuencia se pone a la defensiva o se muestra reservado cuando le preguntan qué hace en línea?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_10	¿Con qué frecuencia ha descubierto a su hijo navegando en línea contra su voluntad?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_11	¿Con qué frecuencia pasa su hijo tiempo en su habitación jugando con la computadora?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_12	¿Con qué frecuencia recibe su hijo llamadas extrañas de nuevos amigos "en línea"?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_13	¿Con qué frecuencia su hijo/a se enfada, grita o se muestra molesto/a si lo molestan mientras está conectado/a?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_14	¿Con qué frecuencia su hijo/a parece más cansado/a y fatigado/a que antes de la llegada de internet?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_15	¿Con qué frecuencia su hijo/a parece preocupado/a por volver a estar conectado/a cuando no está conectado/a?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_16	¿Con qué frecuencia su hijo/a hace rabietas cuando usted interfiere con el tiempo que pasa conectado/a?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_17	¿Con qué frecuencia su hijo/a prefiere pasar tiempo conectado/a en lugar de

		dedicarse a aficiones o intereses que antes disfrutaba?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_18	¿Con qué frecuencia su hijo/a se enfada o se muestra agresivo/a cuando le impone límites de tiempo en línea?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_19	¿Con qué frecuencia su hijo/a prefiere pasar más tiempo conectado/a que saliendo con sus amigos?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_20	¿Con qué frecuencia su hijo/a se siente deprimido/a, malhumorado/a o nervioso/a cuando no está conectado/a, y parece que desaparece al volver a conectarse?
Test de adicción a internet entre padres e hijos	PCIAT-PCIAT_Total	Puntuación total
Escala de Trastornos del Sueño	SDS-Season	Temporada de participación
Escala de Trastornos del Sueño	SDS-SDS_Total_Raw	Puntuación bruta total
Escala de Trastornos del Sueño	SDS-SDS_Total_T	Puntuación T total
Uso de Internet	PreInt_EduHx-Season	Temporada de participación
Uso de Internet	PreInt_EduHx-computerinternet_hoursday	Horas de uso de la computadora/internet

A continuación, vamos a definir los diferentes instrumentos con los cuales se realizaron las mediciones de las investigación, por grupos, para tener una mayor idea a lo que se refieren. Estos instrumentos son:

- **Demográficos** - Información sobre la edad y sexo de los participantes.
- **Uso de Internet** - Número de horas de uso de computadora/internet por día.
- **Escala de evaluación global para niños** - Escala numérica utilizada por los médicos de salud mental para evaluar el funcionamiento general de los jóvenes menores de 18 años.
- **Medidas físicas** - Recopilación de datos de presión arterial, frecuencia cardíaca, altura, peso y medidas de cintura y cadera.
- **FitnessGram Vitals y cinta de correr** - Mediciones de la aptitud cardiovascular evaluadas mediante el protocolo de cinta rodante NHANES.
- **FitnessGram Niño** - Evaluación de la aptitud física relacionada con la salud que mide cinco parámetros diferentes que incluyen capacidad aeróbica, fuerza muscular, resistencia muscular, flexibilidad y composición corporal.
- **Análisis de impedancia bioeléctrica** - Medición de elementos clave de la composición corporal, incluido el IMC, la grasa, los músculos y el contenido de agua.

- **Cuestionario de actividad física** - Información sobre la participación de los niños en actividades vigorosas durante los últimos 7 días.
- **Escala de alteración del sueño**- Escala para categorizar los trastornos del sueño en niños.
- **Actigrafía** - Medida objetiva de la actividad física ecológica a través de un biorastreador de grado de investigación.
- **Prueba de adicción a Internet entre padres e hijos** - Escala de 20 ítems que mide las características y comportamientos asociados con el uso compulsivo de Internet, incluyendo la compulsividad, el escapismo y la dependencia.

Ahora vamos a ver unos archivos de actigrafía `series_train_.parquet` y `series_test_.parquet`, los cuales contienen unos datos de un grupo de participantes que se les dio un acelerómetro para usar durante 30 días continuamente mientras estaban en casa y realizaban sus vidas normales. A continuación los campos y su descripción:

- **id**- El identificador del paciente correspondiente al id campo en `train/test.csv` .
- **step**- Un paso de tiempo entero para cada observación dentro de una serie.
- **X, Y, Z**- Medida de la aceleración, en g , experimentada por el reloj de pulsera a lo largo de cada eje estándar.
- **Enmo** - Según los cálculos y la descripción del paquete `dolly` , ENMO es la Norma Euclidiana Menos Uno de todas las señales del acelerómetro (a lo largo de los ejes x, y, z, medidas en fuerza g) con valores negativos redondeados a cero. Los valores cero indican periodos de inactividad. Si bien no existe una medida estándar de aceleración en este ámbito, esta es una de las características comúnmente calculadas.
- **anglez**- Según lo calculado y descrito por el paquete `dolly` , Angle-Z es una métrica derivada de los componentes individuales del acelerómetro y se refiere al ángulo del brazo con respecto al plano horizontal.
- **non-wear_flag**- Una bandera (0: se está usando el reloj, 1: no se está usando el reloj) para ayudar a determinar los períodos en los que se ha quitado el reloj, según la definición GGIR , que utiliza la desviación estándar y el rango de los datos del acelerómetro.
- **Light** - Medición de la luz ambiental en lux . Consulte aquí para más detalles.
- **battery_voltage**- Una medida del voltaje de la batería en mV .
- **time_of_day**- Hora del día que representa el inicio de una ventana de 5 segundos en la que se han muestreado los datos, con formato %H:%M:%S.%f.

- **weekday**- El día de la semana, codificado como un número entero donde 1 es lunes y 7 es domingo.
- **quarter**- El cuarto del año, un número entero del 1 al 4.
- **relative_date_PCIAT**- El número de días (entero) desde que se administró la prueba PCIAT (los días negativos indican que los datos de actigrafía se recopilaron antes de que se administrara la prueba).

Los archivos con los campos descritos más arriba fueron los utilizados en este proyecto. Adicional a eso se fueron creando algunos necesarios para realizar un estudio óptimo. A continuación, enlace del conjunto de datos en Kaggle:

<https://www.kaggle.com/competitions/child-mind-institute-problematic-internet-use/data>

Entorno de desarrollo:

En este proyecto utilicé el notebook proporcionado por Kaggle a razón que los archivos utilizados eran muy pesados y quería aprovechar esta oportunidad para familiarizarme con este entorno. El propósito es, a futuro, participar en competencias que me permitan seguir adquiriendo experiencias para mi desarrollo profesional como científico de datos.

Ahora mencionaremos las librerías utilizadas en este proyecto:

- **Numpy** : librería base para operaciones numéricas y matriciales de alto rendimiento. Fundamental para cálculos científicos.
- **Pandas**: estructuras de datos (DataFrame, Series) para manipular, limpiar, analizar y explorar datos tabulares.
- **Pyarrow.parquet** : lectura y escritura de archivos en formato Parquet, eficiente para grandes volúmenes de datos columnar.
- **Dask.dataframe** :versión paralela de pandas para trabajar con datasets que no caben en memoria. Ideal para escalabilidad.
- **os** Interacción con el sistema operativo: manejo de rutas, archivos, directorios, etc.
- **Datetime**: manipulación de fechas y tiempos (útil para series temporales, registros, etc)
- **Seaborn**: gráficos estadísticos de alto nivel sobre pandas. Ideal para detectar patrones y relaciones.
- **Matplotlib** :biblioteca de gráficos de bajo nivel, muy flexible. Base para otras librerías como seaborn.

- **Plotly**: visualización interactiva en web (gráficos dinámicos, zoom, hover, etc.).
- **Tqdm** : barra de progreso para loops (por ejemplo, en procesamiento de datos o entrenamiento de modelos).
- **Graphviz, Export_graphviz, Image**: visualización de árboles de decisión entrenados con sklearn.
- **Label_binarize**: conversión de etiquetas multiclase a formato binarizado (para ROC AUC multiclase).
- **LabelEncoder, StandardScaler, MinMaxScaler**: codificación de variables categóricas y escalado de variables numéricas.
- **SimpleImputer, KNNImputer**: imputación de valores faltantes: por media, mediana o vecinos más cercanos.
- **SMOTE**: aumento de datos sintéticos para balancear clases desbalanceadas.
- **Ridge, RidgeCV, Lasso, LassoCV**: regresión lineal regularizada (L2 para Ridge, L1 para Lasso), CV incluye validación cruzada automática de hiperparámetros.
- **LogisticRegression**: algoritmo de clasificación binaria o multiclase basada en regresión logística.
- **LinearRegression**: modelo de regresión lineal sin regularización.
- **DecisionTreeRegressor**: modelo de árbol de decisión para tareas de regresión.
- **RandomForestClassifier, RandomForestRegressor**: algoritmo ensambles de árboles para clasificación o regresión. Robustos y versátiles.
- **XGBClassifier**: algoritmo de clasificación basado en boosting (XGBoost), muy eficiente y ganador frecuente en competencias.
- **MLPClassifier**: algoritmo perceptrón multicapa: red neuronal clásica para clasificación.
- **KNeighborsClassifier**: algoritmo basado en vecinos más cercanos para clasificación.
- **KMeans**: algoritmo de agrupamiento no supervisado.
- **PCA**: algoritmo de reducción de dimensionalidad mediante análisis de componentes principales.

- **Accuracy_score, Precision_score, Recall_score, Confusion_matrix, Classification_report:** métricas estándar de clasificación.
- **Roc_curve, auc:** curvas ROC y cálculo del área bajo la curva.
- **ConfusionMatrixDisplay:** visualización de matriz de confusión.
- **mean_squared_error, r2_score, mean_absolute_error:** métricas para evaluación de regresión.
- **Cross_validate, GridSearchCV, train_test_split:** validación cruzada, búsqueda de hiperparámetros, y separación de datos de entrenamiento/test.
- **ThreadPoolExecutor:** ejecución paralela de tareas (multithreading). Útil para acelerar ciclos de procesamiento.
- **Scipy.stats:** pruebas estadísticas, generación de números aleatorios, y análisis de distribución.

A modo nota, la mayoría de estas librerías fueron utilizadas en su momento, ya que este proyecto opte por probar muchos modelos y procedimiento que en su momento eran necesarios.

Métodos

Los métodos o procedimientos seguidos y utilizados fueron los siguientes:

Entendimiento del negocio o del tema del proyecto:

- Investigue sobre el uso problemático del internet: consulte artículo, publicación, videos y trabajos previos.

Entendimiento de los datos:

- Se visualizaron, resumieron y describieron los datos

Preprocesamiento de los datos:

- Se crearon y resumieron tablas
- Se crearon columnas nuevas a partir de otras
- Se modificaron los tipos de datos a algunas columnas
- Se trabajaron los valores faltantes
- Se revisaron los valores duplicados
- Se transformaron valores atípicos
- Se utilizaron modelos no supervisados

Análisis exploratorio de los datos:

- Se visualizaron datos para ver tendencias, patrones y encontrar algunos hallazgos
- Análisis Univariante
- Análisis Bivariante
- Análisis de correlación

Modelización de los datos:

- Se entrenaron y entrenaron varios modelos para ver cuál era el más eficiente tenía mejor rendimiento

Validación de los modelos:

- Se probaron los modelos con diferentes métricas
- Se selecciono aquel que tenía mejor rendimiento

Resultado

Librerías

```
import numpy as np
import pyarrow.parquet as pq
import dask.dataframe as dd # Se utiliza este modulo debido para trabajar con archivos grandes.
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from tqdm import tqdm
import plotly.express as px
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, RidgeCV, Lasso, LassoCV
from concurrent.futures import ThreadPoolExecutor
from imblearn.over_sampling import SMOTE # Se utiliza para crear datos sinteticos. Especialmente cuando existen desbalances en los datos.
from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder, MinMaxScaler
from sklearn.impute import SimpleImputer #Utilizar este modulo para imputar los valores faltantes de las mediciones
from sklearn.impute import KNNImputer #Utilizar este modulo para imputar los valores faltantes de las mediciones.
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, ConfusionMatrixDisplay, classification_report
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV, cross_validate
from sklearn.linear_model import LogisticRegression, LinearRegression
import xgboost as xgb
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from scipy.stats import chi2_contingency, randint, probplot
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.tree import export_graphviz
from IPython.display import Image
import graphviz
import sklearn.metrics as metrics
from datetime import datetime
```

Estas son las librerías que se utilizaron en el proyecto. Durante la ejecución de este se probaron diferentes modelos y métodos, los cuales algunos se descartaron.

Funciones propias

```
def convertir_columnas_categoricas_0(columna):
    for col in columna:
        if col == "PAQ_A-Season":
            continue
        elif df_cluster_0[col].dtype == 'category':
            moda = df_cluster_0[col].value_counts().idxmax()
            df_cluster_0.loc[:, col] = df_cluster_0[col].fillna(modas)
        else:
            print(f"Revisar la conversión {col}")

def convertir_columnas_categoricas_1(columna):
    for col in columna:
        if df_cluster_1[col].dtype == 'category':
            moda = df_cluster_1[col].value_counts().idxmax()
            df_cluster_1.loc[:, col] = df_cluster_1[col].fillna(modas)
        else:
            print(f"Revisar la conversión {col}")

def convertir_columnas_categoricas_2(columna):
    for col in columna:
        if df_cluster_2[col].dtype == 'category':
            moda = df_cluster_2[col].value_counts().idxmax()
            df_cluster_2.loc[:, col] = df_cluster_2[col].fillna(modas)
        else:
            print(f"Revisar la conversión {col}")

def convertir_a_categoricas(columna):
    for col in columna:
        df_completo[col] = df_completo[col].astype('category')
```

```

def convertir_a_categoricas_0(columna):
    for col in columna:
        df_cluster_0[col] = df_cluster_0[col].astype('category')

def convertir_a_categoricas_1(columna):
    for col in columna:
        df_cluster_1[col] = df_cluster_1[col].astype('category')

def convertir_a_categoricas_2(columna):
    for col in columna:
        df_cluster_2[col] = df_cluster_2[col].astype('category')

def reemplazar_atipicos_con_mediana_0(columna):

    for col in columna:
        mediana = df_cluster_0[col].median()

        # Calcular los límites usando el rango intercuartílico (IQR)
        Q1 = df_cluster_0[col].quantile(0.25)
        Q3 = df_cluster_0[col].quantile(0.75)
        IQR = Q3 - Q1
        limite_inferior = Q1 - 1.5 * IQR
        limite_superior = Q3 + 1.5 * IQR

        lista = [ dato for dato in df_cluster_0[col] if dato < limite_inferior or dato > limite_superior]

        df_cluster_0[col] = df_cluster_0[col].replace(lista, mediana)

def reemplazar_atipicos_con_mediana_1(columna):

    for col in columna:
        mediana = df_cluster_1[col].median()

        # Calcular los límites usando el rango intercuartílico (IQR)
        Q1 = df_cluster_1[col].quantile(0.25)
        Q3 = df_cluster_1[col].quantile(0.75)
        IQR = Q3 - Q1
        limite_inferior = Q1 - 1.5 * IQR
        limite_superior = Q3 + 1.5 * IQR

        lista = [ dato for dato in df_cluster_1[col] if dato < limite_inferior or dato > limite_superior]

        df_cluster_1[col] = df_cluster_1[col].replace(lista, mediana)

def reemplazar_atipicos_con_mediana_2(columna):

    for col in columna:
        mediana = df_cluster_2[col].median()

        # Calcular los límites usando el rango intercuartílico (IQR)
        Q1 = df_cluster_2[col].quantile(0.25)
        Q3 = df_cluster_2[col].quantile(0.75)
        IQR = Q3 - Q1
        limite_inferior = Q1 - 1.5 * IQR
        limite_superior = Q3 + 1.5 * IQR

        lista = [ dato for dato in df_cluster_2[col] if dato < limite_inferior or dato > limite_superior]

        df_cluster_2[col] = df_cluster_2[col].replace(lista, mediana)

```

```
def sustitucion_valores_nulos_num(columna, df):
    for col in columna:
        if df[col].dtype == 'int' or df[col].dtype == 'float':
            mediana = df[col].median()
            df.loc[:, col] = df[col].fillna(mediana)
        else:
            print(f"Revisar la conversión {col}")

def sustitucion_valores_nulos_num_p(columna, df):
    for col in columna:
        if df[col].dtype == 'int' or df[col].dtype == 'float':
            mediana = df[col].median()
            df.loc[:, col] = df[col].fillna(mediana)
        else:
            print(f"Revisar la conversión {col}")

def imputar_con_KNN(columnas, df):
    columnas_a_imputar = columnas
    subconjunto = df[columnas_a_imputar]

    # Aplicar KNNImputer solo a las columnas seleccionadas
    knn_imputer = KNNImputer(n_neighbors=5)
    subconjunto_imputado = pd.DataFrame(knn_imputer.fit_transform(subconjunto), columns=columnas_a_imputar)

    # Reemplazar las columnas imputadas en el DataFrame original
    df[columnas_a_imputar] = subconjunto_imputado
```

```
def imputar_con_SI(columnas, df):
    # Definir las columnas a imputar
    columnas_a_imputar_GP = columnas

    # Comprobar si las columnas existen en el DataFrame
    columnas_existentes = [col for col in columnas_a_imputar_GP if col in df.columns]

    # Seleccionar solo las columnas existentes
    subconjunto_GP = df[columnas_existentes]

    # Imputar valores faltantes usando la media
    simple_imputer = SimpleImputer(strategy='mean')
    subconjunto_imputado_GP = pd.DataFrame(simple_imputer.fit_transform(subconjunto_GP), columns=columnas_existentes, index=df.index)

    # Actualizar las columnas imputadas en el DataFrame original
    df[columnas_existentes] = subconjunto_imputado_GP

    return df

# Esa función se utiliza para crear un histograma de los modelos de clasificación para saber que tan bien clasifica
def represento_doble_hist(x_1, x_0, n_bins=11, title='', label_1='Clase 1',
                          label_0='Clase 0', density=0, figsize=(4, 3)):
    bins = n_bins
    plt.figure(figsize=figsize) # Tamaño más pequeño por gráfico
    plt.hist(x_1, bins, density=density, alpha=0.5, label=label_1, color='red')
    plt.hist(x_0, bins, density=density, alpha=0.5, label=label_0, color='green')
    plt.title(title)
    plt.xlabel('Probabilidad predicha')
    plt.ylabel('Frecuencia relativa' if density else 'Frecuencia')
    plt.legend(loc='best')
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

```
# Esta función se utiliza para crear varios graficos histogramas utilizando la estrategia OvR
def representar_histogramas_ovr(Y_test, y_t_pred_pro, clases=None, n_bins=20, figsize=(4, 3)):
    n_clases = y_t_pred_pro.shape[1]
    if clases is None:
        clases = list(range(n_clases))

    for i in clases:
        y_scores = y_t_pred_pro[:, i]
        x_1 = y_scores[Y_test == i]
        x_0 = y_scores[Y_test != i]

        titulo = f'Distribución de probabilidades - Clase {i} vs Resto'

        represento_doble_hist(x_1, x_0, n_bins=n_bins,
                              title=titulo,
                              label_1=f'Clase {i} (positivos)',
                              label_0='Resto (negativos)',
                              density=1,
                              figsize=figsize)
```

Estas funciones fueron creadas para diferentes motivos: convertir variables categóricas, reemplazar valores nulos, imputar valores atípicos, representación de histogramas entre otros.

Entendimiento y procesamiento de datos: dataset series

```
# Conjunto de datos de Actigrafia

df_series_train = dd.read_parquet('/kaggle/input/child-mind-institute-problematic-internet-use/series_train.parquet')
df_series_test = dd.read_parquet('/kaggle/input/child-mind-institute-problematic-internet-use/series_test.parquet')
```

Se esta cargando el conjunto de la actigrafia provenientes del grupo de participantes que fueron seleccionados para medir sus actividades diarias.

```
# Tratamiento para cargar y hacer el resumen de la columna dias de la semana
conteo_por_grupo = df_series_train.groupby("id")["weekday"].value_counts().compute()

df_result = conteo_por_grupo.reset_index(name='conteo')
valores_mas_frecuentes = df_result.loc[df_result.groupby('id')['conteo'].idxmax()]

valores_mas_frecuentes = valores_mas_frecuentes[["id", "weekday"]].drop_duplicates()

df_weekday = valores_mas_frecuentes.dropna()
```

```
# Tratamiento para cargar y hacer el resumen la columna temporadas del año

conteo_por_grupo = df_series_train.groupby("id")["quarter"].value_counts().compute()

df_result = conteo_por_grupo.reset_index(name='conteo')
valores_mas_frecuentes = df_result.loc[df_result.groupby('id')['conteo'].idxmax()]

valores_mas_frecuentes = valores_mas_frecuentes[["id", "quarter"]].drop_duplicates()

df_quarter = valores_mas_frecuentes.dropna()
```

Ahora estamos realizando un tipo de mantenimiento a las columnas quarter y weekday de la tabla series_train_parquet para poder anexarla en un resumen de este mismo dataset.

```
# Conteo de la cantidad de filas del conjunto de datos

cantidad_filas = df_series_train.shape[0].compute()
print(f"Cantidad de filas: {cantidad_filas}")
```

```
Cantidad de filas: 314569149
```

Aquí podemos ver la cantidad de registros que tiene este dataset. A raíz de eso decidimos realizar un resumen esos con un promedio por participantes en cada de una de las columnas para que sea más manejable.

```
# Realizando un resumen de la data series_train debido a que es muy pesada, tiene mas de 314 millones de registros

resumen = df_series_train.groupby('id').agg({
    'X': 'mean',
    'Y': 'mean',
    'Z': 'mean',
    'enmo': 'mean',
    'light': 'mean',
    'battery_voltage': 'mean',
    'time_of_day': 'mean',
    'relative_date_PCIAT': 'mean'
})
```

Con este código se realizó la tabla resumen que contiene el promedio de cada una de las columnas de los participantes.

```
print(resumen.compute())
```

id	X	Y	Z	enmo	light	battery_voltage	\
0417c91e	-0.147090	-0.017002	-0.004732	0.058441	123.444945	3922.278551	
1c063ebc	0.400091	-0.069567	-0.035199	0.044230	89.860481	3848.648695	
2cda86bd	0.027417	-0.384444	-0.278186	0.005275	3.562555	3839.013173	
350cf38b	-0.241119	-0.001396	-0.188692	0.048448	21.641680	3905.451498	
64ed48c4	-0.009472	-0.028348	-0.964124	0.000434	9.016723	3825.847369	
...	...	...	...	...	...	...	...
fe9c71d8	NaN	NaN	NaN	NaN	NaN	NaN	NaN
fecc07d6	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ff18b749	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ffcd4dbd	NaN	NaN	NaN	NaN	NaN	NaN	NaN
ffed1dd5	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
# Revisando los valores faltantes del resumen
valores_faltantes_por_columna = resumen.isna().sum().compute()
print(valores_faltantes_por_columna)
```

```
X          60758
Y          60758
Z          60758
enmo       60758
light       60758
battery_voltage 60758
time_of_day 60758
relative_date_PCIAT 60758
dtype: int64
```

```
# Eliminando los valores faltantes generados por el resumen
resumen_limpio = resumen.dropna()
```

Aquí estamos revisando los valores nulos de la tabla resumen que aparecieron en la primera consulta luego de crearla. Cuando le aplicamos los métodos `isna()` y `sum()` para

que os arroje una suma de la cantidad de valores nulos con un total 60,758 casos nulos. A raíz de eso aplicamos el método `dropna()` para eliminarlos.

```
# Eliminando los valores duplicados de la tabla resumen
resumen_limpio = resumen_limpio.drop_duplicates()
```

```
# Reseteo de los indices debido al mantenimiento previo
resumen_limpio = resumen_limpio.reset_index()

resumen_limpio = resumen_limpio.compute()
```

Luego de eso aplicamos el método `drop_duplicates()` para eliminar los valores y luego reseteamos los índices luego del mantenimiento previo.

```
# Uniendo las columnas weekday y quarter
df_union_c = pd.merge(df_weekday, df_quarter, on='id', how='inner')
```

```
# Uniendo los la tabla resumen luego de su tratamiento y la tabla unión de las columnas weekday y quarter
df_union_series = pd.merge(resumen_limpio, df_union_c, on='id', how='left')
```

Luego se creo la tabla `df_union_c` combinando las tablas `df_weekday` y `df_quarter` que habíamos creado. Luego de hacer esto creamos una unión con la tabla `resumen_limpio` ggpara tener la tabla completa de `series_train.parquet`

```
# Revisando la unión
df_union_series.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 994 entries, 0 to 993
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    994 non-null   category
1   X                    994 non-null   float64
2   Y                    994 non-null   float64
3   Z                    994 non-null   float64
4   enmo                 994 non-null   float64
5   light                994 non-null   float64
6   battery_voltage      994 non-null   float64
7   time_of_day          994 non-null   float64
8   relative_date_PCIAT  994 non-null   float64
9   weekday              994 non-null   category
10  quarter              994 non-null   category
dtypes: category(3), float64(8)
memory usage: 106.5 KB
```

Aplicamos el `info()` al nuevo dataframe para ver la cantidad de registros, los tipos de datos y las columnas que contenía.

```
# Cargando los datos tabulares sobre las mediciones con intruementos.
test = pd.read_csv("/kaggle/input/child-mind-institute-problematic-internet-use/test.csv")
train = pd.read_csv("/kaggle/input/child-mind-institute-problematic-internet-use/train.csv")
```

Luego de eso aplicamos cargamos el conjunto de datos `test.csv` y `train.csv` que contenían los daos de los instrumentos.

```
# Revisado las primeras 5 filas
train.head()
```

	Id	Basic_Demos-Enroll_Season	Basic_Demos-Age	Basic_Demos-Sex	CGAS-Season	CGAS-Score	Physical-Season	Physical-BMI	Physical-Height	Physical-Weight	PCIAT-PCIAT_18	PCIAT-PCIAT_19	PCIAT-PCIAT_20	PCIAT-PCIAT_Total	SDS-Season	SDS-SDS_Total_Raw	SDS-SDS_Total_T	PreInt_EduHx-Season	PreInt_EduHx-computerInternet_hoursday	all	
0	00008f9	Fall	5	0	Winter	51.0	Fall	16.877316	46.0	50.8	...	4.0	2.0	4.0	55.0	NaN	NaN	NaN	Fall	3.0	2.0
1	000f5460	Summer	9	0	NaN	NaN	Fall	14.035590	48.0	46.0	...	0.0	0.0	0.0	0.0	Fall	46.0	64.0	Summer	0.0	0.0
2	00105258	Summer	10	1	Fall	71.0	Fall	16.648696	56.5	75.6	...	2.0	1.0	1.0	28.0	Fall	38.0	54.0	Summer	2.0	0.0
3	00115b9f	Winter	9	0	Fall	71.0	Summer	18.292347	56.0	81.6	...	3.0	4.0	1.0	44.0	Summer	31.0	45.0	Winter	0.0	1.0
4	0016bb22	Spring	18	1	Summer	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

5 rows × 22 columns

Aplicamos el método `head()` a la tabla `train` para ver los primeros 5 registros.

```
#Revisando si tiene valores duplicados
train.duplicated().sum()
```

0

Revisamos si tiene valores duplicados y vemos que no.

```
# Uniendo los conjuntos de datos train y df_union_series

df_completo = pd.merge(train,df_union_series, on='id',how='left')
```

Luego hacemos una unión entre la tabla `train` y `df_union_series` para completa nuestra tabla que contenga los datos de las mediciones de los instrumentos y la actigrafía.

```
#Revisando las dimensiones del conjunto de datos
df_completo.shape
```

(3960, 92)

Aquí vemos la forma del conjunto datos nuevo, con 3960 registros y 92 columnas.

```
# Revisando los tipos de datos y valores nulos del conjunto de datos

df_completo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3960 entries, 0 to 3959
Data columns (total 92 columns):
```

#	Column	Non-Null Count	Dtype
0	id	3960 non-null	object
1	Basic_Demos-Enroll_Season	3960 non-null	object
2	Basic_Demos-Age	3960 non-null	int64
3	Basic_Demos-Sex	3960 non-null	int64
4	CGAS-Season	2555 non-null	object
5	CGAS-CGAS_Score	2421 non-null	float64
6	Physical-Season	3310 non-null	object
7	Physical-BMI	3022 non-null	float64
8	Physical-Height	3027 non-null	float64
9	Physical-Weight	3076 non-null	float64
10	Physical-Waist_Circumference	898 non-null	float64
11	Physical-Diastolic_BP	2954 non-null	float64
12	Physical-HeartRate	2967 non-null	float64
13	Physical-Systolic_BP	2954 non-null	float64
14	Fitness_Endurance-Season	1308 non-null	object
15	Fitness_Endurance-Max_Stage	743 non-null	float64
16	Fitness_Endurance-Time_Mins	740 non-null	float64
17	Fitness_Endurance-Time_Sec	740 non-null	float64
18	FGC-Season	3346 non-null	object
19	FGC-FGC_CU	2322 non-null	float64
20	FGC-FGC_CU_Zone	2282 non-null	float64
21	FGC-FGC_GSND	1074 non-null	float64
22	FGC-FGC_GSND_Zone	1062 non-null	float64
23	FGC-FGC_GSD	1074 non-null	float64
24	FGC-FGC_GSD_Zone	1063 non-null	float64
25	FGC-FGC_PU	2310 non-null	float64
26	FGC-FGC_PU_Zone	2271 non-null	float64
27	FGC-FGC_SRL	2305 non-null	float64
28	FGC-FGC_SRL_Zone	2267 non-null	float64
29	FGC-FGC_SRR	2307 non-null	float64
30	FGC-FGC_SRR_Zone	2269 non-null	float64
31	FGC-FGC_TL	2324 non-null	float64
32	FGC-FGC_TL_Zone	2285 non-null	float64
33	BIA-Season	2145 non-null	object
34	BIA-BIA_Activity_Level_num	1991 non-null	float64
35	BIA-BIA_BMC	1991 non-null	float64
36	BIA-BIA_BMI	1991 non-null	float64
37	BIA-BIA_BMR	1991 non-null	float64
38	BIA-BIA_DEE	1991 non-null	float64
39	BIA-BIA_ECW	1991 non-null	float64
40	BIA-BIA_FFM	1991 non-null	float64
41	BIA-BIA_FFMI	1991 non-null	float64
42	BIA-BIA_FMI	1991 non-null	float64
43	BIA-BIA_Fat	1991 non-null	float64
44	BIA-BIA_Frame_num	1991 non-null	float64
45	BIA-BIA_ICW	1991 non-null	float64
46	BIA-BIA_LDM	1991 non-null	float64
47	BIA-BIA_LST	1991 non-null	float64
48	BIA-BIA_SMM	1991 non-null	float64
49	BIA-BIA_TBW	1991 non-null	float64
50	PAO A-Season	475 non-null	object

```

51 PAQ_A-PAQ_A_Total          475 non-null    float64
52 PAQ_C-Season               1721 non-null   object
53 PAQ_C-PAQ_C_Total          1721 non-null   float64
54 PCIAT-Season               2736 non-null   object
55 PCIAT-PCIAT_01             2733 non-null   float64
56 PCIAT-PCIAT_02             2734 non-null   float64
57 PCIAT-PCIAT_03             2731 non-null   float64
58 PCIAT-PCIAT_04             2731 non-null   float64
59 PCIAT-PCIAT_05             2729 non-null   float64
60 PCIAT-PCIAT_06             2732 non-null   float64
61 PCIAT-PCIAT_07             2729 non-null   float64
62 PCIAT-PCIAT_08             2730 non-null   float64
63 PCIAT-PCIAT_09             2730 non-null   float64
64 PCIAT-PCIAT_10             2733 non-null   float64
65 PCIAT-PCIAT_11             2734 non-null   float64
66 PCIAT-PCIAT_12             2731 non-null   float64
67 PCIAT-PCIAT_13             2729 non-null   float64
68 PCIAT-PCIAT_14             2732 non-null   float64
69 PCIAT-PCIAT_15             2730 non-null   float64
70 PCIAT-PCIAT_16             2728 non-null   float64
71 PCIAT-PCIAT_17             2725 non-null   float64
72 PCIAT-PCIAT_18             2728 non-null   float64
73 PCIAT-PCIAT_19             2730 non-null   float64
74 PCIAT-PCIAT_20             2733 non-null   float64
75 PCIAT-PCIAT_Total          2736 non-null   float64
76 SDS-Season                 2618 non-null   object
77 SDS-SDS_Total_Raw          2609 non-null   float64
78 SDS-SDS_Total_T            2606 non-null   float64
79 PreInt_EduHx-Season        3540 non-null   object
80 PreInt_EduHx-computerinternet hoursday 3301 non-null   float64

81 sii                        2736 non-null   float64
82 X                          994 non-null   float64
83 Y                          994 non-null   float64
84 Z                          994 non-null   float64
85 enmo                       994 non-null   float64
86 light                      994 non-null   float64
87 battery_voltage            994 non-null   float64
88 time_of_day                 994 non-null   float64
89 relative_date_PCIAT        994 non-null   float64
90 weekday                     994 non-null   category
91 quarter                     994 non-null   category
dtypes: category(2), float64(76), int64(2), object(12)
memory usage: 2.7+ MB

```

Aquí vemos columnas, el tipo de datos y la cantidad de valores no nulos que tiene el nuevo conjunto de datos. Aquí vemos que muchas columnas tienen muchos valores nulos por lo que se utilizaron dos procesos para mejorar esta situación con los valores nulos.

```

columnas_categoricas = ['Basic-Demos-Sex', 'FGC-FGC_CU_Zone', 'FGC-FGC_GSND_Zone', 'FGC-FGC_GSD_Zone', 'FGC-FGC_PU_Zone', 'FGC-FGC_SRL_Zone', 'FGC-FGC_SRR_Zone',
'FGC-FGC_TL_Zone', 'BIA-BIA_Activity_Level_num', 'BIA-BIA_Frame_num', 'PCIAT-PCIAT_01', 'PCIAT-PCIAT_02', 'PCIAT-PCIAT_03', 'PCIAT-PCIAT_04',
'PCIAT-PCIAT_05', 'PCIAT-PCIAT_06', 'PCIAT-PCIAT_07', 'PCIAT-PCIAT_08', 'PCIAT-PCIAT_09', 'PCIAT-PCIAT_10', 'PCIAT-PCIAT_11',
'PCIAT-PCIAT_12', 'PCIAT-PCIAT_13', 'PCIAT-PCIAT_14', 'PCIAT-PCIAT_15', 'PCIAT-PCIAT_16', 'PCIAT-PCIAT_17', 'PCIAT-PCIAT_18',
'PCIAT-PCIAT_19', 'PCIAT-PCIAT_20', 'PreInt_EduHx-computerinternet_hoursday', 'quarter', 'weekday']

```

```

columnas_str = ['Basic-Demos-Enroll_Season', 'CGAS-Season', 'Physical-Season', 'Fitness_Endurance-Season',
'FGC-Season', 'BIA-Season', 'PAQ_A-Season', 'PAQ_C-Season', 'PCIAT-Season', 'SDS-Season', 'PreInt_EduHx-Season']

```

Se procedió a crear dos listas con las columnas del tipo float64 y object para convertirlas en categóricas, la cual era su verdadero tipo.

```
# Transformando algunas columnas a categoricas
convertir_a_categoricas(columnas_categoricas)
```

Aplicamos este método para convertirlas las variables al tipo category.

```
# Revisando los cambios
df_completo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3960 entries, 0 to 3959
Data columns (total 92 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         3960 non-null   object
1   Basic_Demos-Enroll_Season                3960 non-null   object
2   Basic_Demos-Age                          3960 non-null   int64
3   Basic_Demos-Sex                          3960 non-null   category
4   CGAS-Season                              2555 non-null   object
5   CGAS-CGAS_Score                           2421 non-null   float64
6   Physical-Season                           3310 non-null   object
7   Physical-BMI                             3022 non-null   float64
8   Physical-Height                           3027 non-null   float64
9   Physical-Weight                           3076 non-null   float64
10  Physical-Waist_Circumference               898 non-null    float64
11  Physical-Diastolic_BP                     2954 non-null   float64
12  Physical-HeartRate                         2967 non-null   float64
13  Physical-Systolic_BP                      2954 non-null   float64
14  Fitness_Endurance-Season                  1308 non-null   object
15  Fitness_Endurance-Max_Stage                743 non-null    float64
16  Fitness_Endurance-Time_Mins                740 non-null    float64
17  Fitness_Endurance-Time_Sec                740 non-null    float64
18  FGC-Season                                3346 non-null   object
19  FGC-FGC_CU                                2322 non-null   float64
20  FGC-FGC_CU_Zone                           2282 non-null   category
21  FGC-FGC_GSND                               1074 non-null   float64
22  FGC-FGC_GSND_Zone                          1062 non-null   category
23  FGC-FGC_GSD                               1074 non-null   float64
24  FGC-FGC_GSD_Zone                          1063 non-null   category
25  FGC-FGC_PU                                2310 non-null   float64
```

26	FGC-FGC_PU_Zone	2271	non-null	category
27	FGC-FGC_SRL	2305	non-null	float64
28	FGC-FGC_SRL_Zone	2267	non-null	category
29	FGC-FGC_SRR	2307	non-null	float64
30	FGC-FGC_SRR_Zone	2269	non-null	category
31	FGC-FGC_TL	2324	non-null	float64
32	FGC-FGC_TL_Zone	2285	non-null	category
33	BIA-Season	2145	non-null	object
34	BIA-BIA_Activity_Level_num	1991	non-null	category
35	BIA-BIA_BMC	1991	non-null	float64
36	BIA-BIA_BMI	1991	non-null	float64
37	BIA-BIA_BMR	1991	non-null	float64
38	BIA-BIA_DEE	1991	non-null	float64
39	BIA-BIA_ECw	1991	non-null	float64
40	BIA-BIA_FFM	1991	non-null	float64
41	BIA-BIA_FFMi	1991	non-null	float64
42	BIA-BIA_FMI	1991	non-null	float64
43	BIA-BIA_Fat	1991	non-null	float64
44	BIA-BIA_Frame_num	1991	non-null	category
45	BIA-BIA_ICw	1991	non-null	float64
46	BIA-BIA_LDM	1991	non-null	float64
47	BIA-BIA_LST	1991	non-null	float64
48	BIA-BIA_SMM	1991	non-null	float64
49	BIA-BIA_TBW	1991	non-null	float64
50	PAQ_A-Season	475	non-null	object
51	PAQ_A-PAQ_A_Total	475	non-null	float64
52	PAQ_C-Season	1721	non-null	object
53	PAQ_C-PAQ_C_Total	1721	non-null	float64
54	PCIAT-Season	2736	non-null	object
55	PCIAT-PCIAT_01	2733	non-null	category
56	PCIAT-PCIAT_02	2734	non-null	category
57	PCIAT-PCIAT_03	2731	non-null	category
58	PCIAT-PCIAT_04	2731	non-null	category
59	PCIAT-PCIAT_05	2729	non-null	category
60	PCIAT-PCIAT_06	2732	non-null	category
61	PCIAT-PCIAT_07	2729	non-null	category
62	PCIAT-PCIAT_08	2730	non-null	category
63	PCIAT-PCIAT_09	2730	non-null	category
64	PCIAT-PCIAT_10	2733	non-null	category
65	PCIAT-PCIAT_11	2734	non-null	category
66	PCIAT-PCIAT_12	2731	non-null	category
67	PCIAT-PCIAT_13	2729	non-null	category
68	PCIAT-PCIAT_14	2732	non-null	category
69	PCIAT-PCIAT_15	2730	non-null	category
70	PCIAT-PCIAT_16	2728	non-null	category
71	PCIAT-PCIAT_17	2725	non-null	category
72	PCIAT-PCIAT_18	2728	non-null	category
73	PCIAT-PCIAT_19	2730	non-null	category
74	PCIAT-PCIAT_20	2733	non-null	category
75	PCIAT-PCIAT Total	2736	non-null	float64

```

76 SDS-Season                2618 non-null    object
77 SDS-SDS_Total_Raw        2609 non-null    float64
78 SDS-SDS_Total_T          2606 non-null    float64
79 PreInt_EduHx-Season       3540 non-null    object
80 PreInt_EduHx-computerinternet_hoursday 3301 non-null    category
81 sii                       2736 non-null    float64
82 X                         994 non-null     float64
83 Y                         994 non-null     float64
84 Z                         994 non-null     float64
85 enmo                     994 non-null     float64
86 light                    994 non-null     float64
87 battery_voltage          994 non-null     float64
88 time_of_day              994 non-null     float64
89 relative_date_PCIAT      994 non-null     float64
90 weekday                  994 non-null     category
91 quarter                  994 non-null     category
dtypes: category(33), float64(46), int64(1), object(12)
memory usage: 1.9+ MB

```

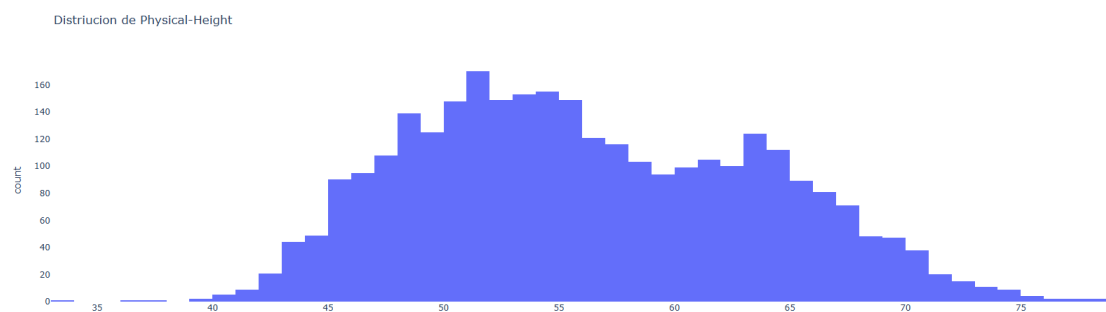
Aquí podemos ver los cambios realizados y los variables convertidas en category.

Distribuciones variables y Outliers

```

fig = px.histogram(df_completo, x = "Physical-Height", title = "Distriucion de Physical-Height" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()

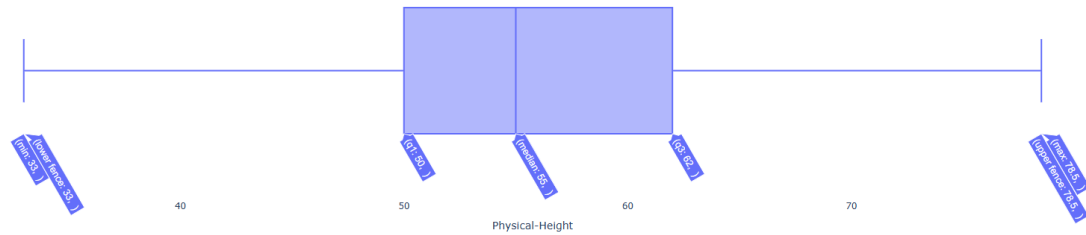
```



```

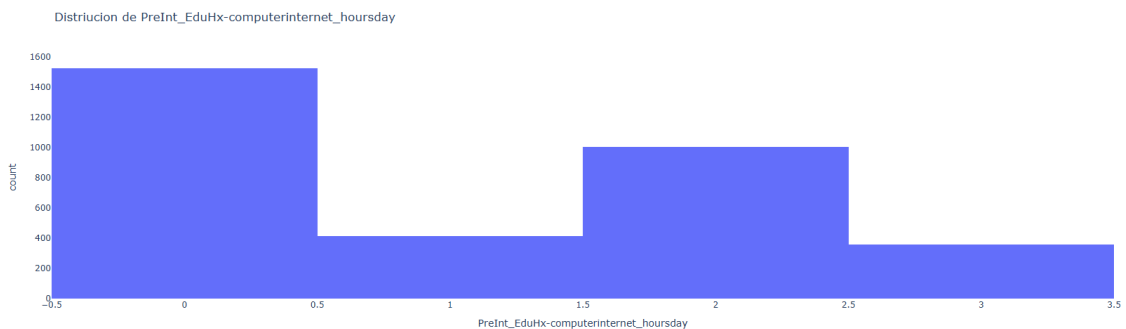
fig = px.box(df_completo, x = "Physical-Height", title = "Valores outliers Physical-Height")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()

```



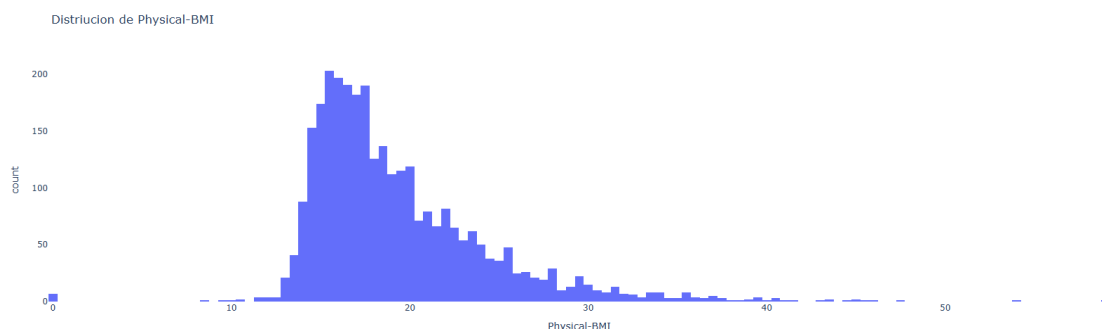
Podemos ver que la mayoría de los participantes están dentro del rango 47 y 67 pulgadas. De acuerdo con estudios la estatura podría ser un indicador indirecto del nivel de desarrollo y podrían ayudar detectar desajustes físicos que a su vez podrían estar relacionados con hábitos sedentarios o uso problemático del internet.

```
fig = px.histogram(df_completo, x = "PreInt_EduHx-computerinternet_hoursday", title = "Distriucion de PreInt_EduHx-computerinternet_hoursday" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



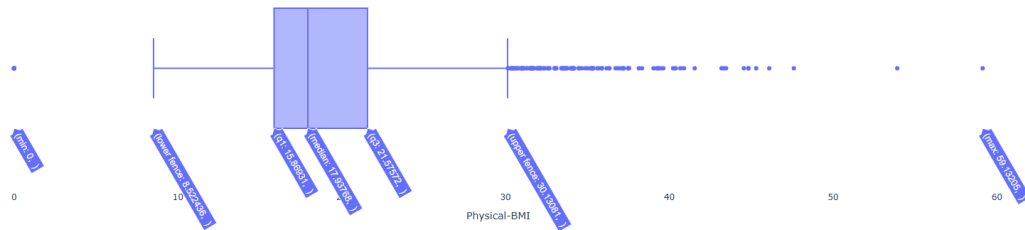
Se puede ver en estos gráficos que un gran grupo de participantes no usa o navega en el internet, esto podría, a mi entender, por la edad o por falta de acceso al mismo.

```
fig = px.histogram(df_completo, x = "Physical-BMI", title = "Distriucion de Physical-BMI" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
fig = px.box(df_completo, x = "Physical-BMI", title = "Valores outliers Physical-BMI")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```

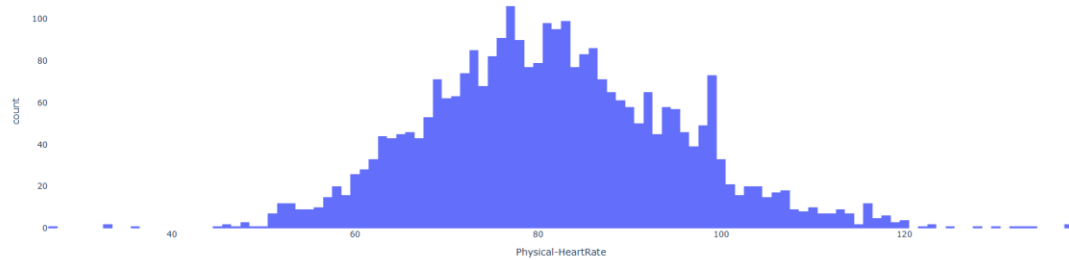
Valores outliers Physical-BMI



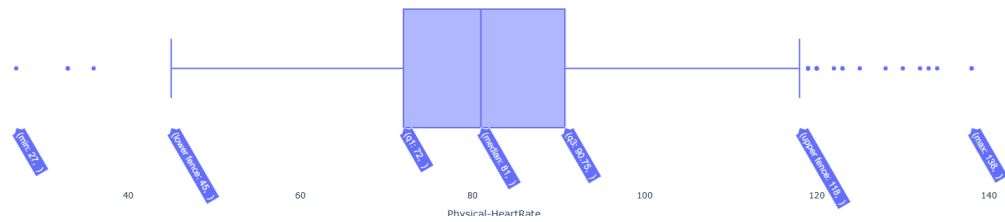
Se puede visualizar que existen valores atípicos, esta columna representa el índice de masa corporal de los participantes. Esta variable podría tener una influencia indirecta cuando se considera con otra variable como el nivel de actividad física.

```
fig = px.histogram(df_completo, x = "Physical-HeartRate", title = "Distriucion de Physical-HeartRate" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```

Distriucion de Physical-HeartRate

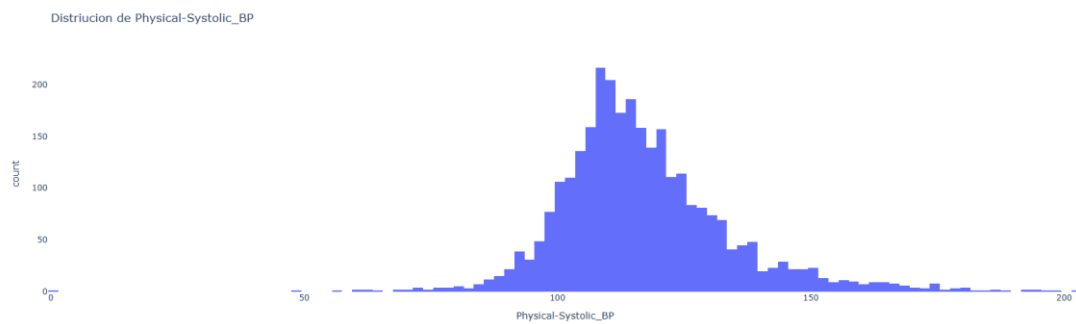


```
fig = px.box(df_completo, x = "Physical-HeartRate", title = "Valores outliers Physical-HeartRate")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```

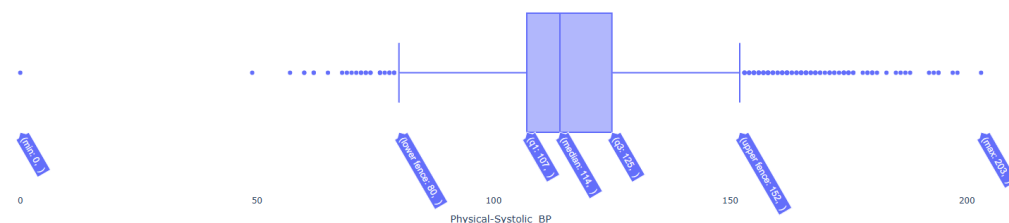


Esta variable representa las frecuencias cardiacas de los participantes. Podemos ver que existen valores nulos los cuales tendremos imputarlos debido a que esta variable, como veremos adelante, esta correlacionado con la variable objetivo.

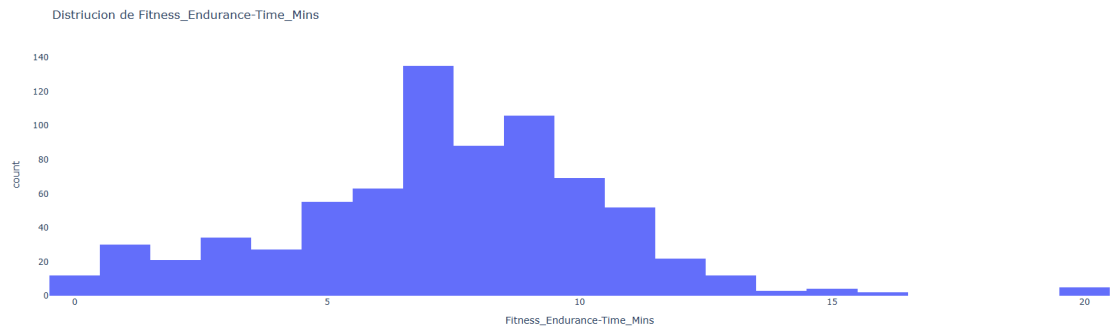
```
fig = px.histogram(df_completo, x = "Physical-Systolic_BP", title = "Distriucion de Physical-Systolic_BP" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



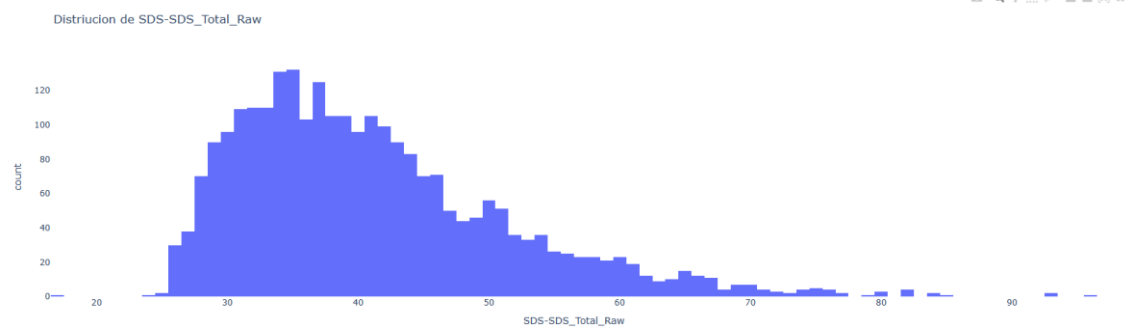
```
fig = px.box(df_completo, x = "Physical-Systolic_BP", title = "Valores outliers Physical-Systolic_BP")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



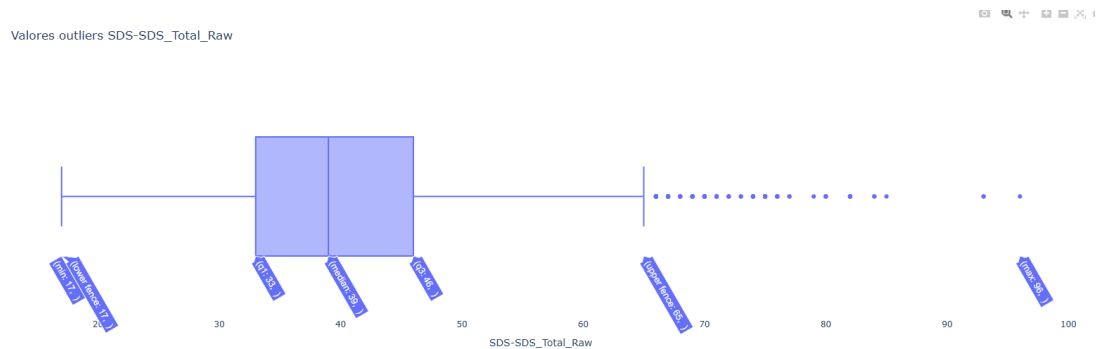
```
fig = px.histogram(df_completo, x = "Fitness_Endurance-Time_Mins", title = "Distriucion de Fitness_Endurance-Time_Mins" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
fig = px.histogram(df_completo, x = "SDS-SDS_Total_Raw", title = "Distriucion de SDS-SDS_Total_Raw" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
fig = px.box(df_completo, x = "SDS-SDS_Total_Raw", title = "Valores outliers SDS-SDS_Total_Raw")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



En estas muestras de las columnas seleccionadas podemos ver que existen valores atípicos los cuales tendremos que tratarlos.

Clústeres: valores atípicos y valores nulos

Las variables que seleccionamos realizar los clústeres fueron la variable sexo y edad. Entiendo que estas dos características serían idóneas para este propósito.

```
df_completo["Basic_Demos-Sex"].astype(int)
```

Primero convertimos la variable sexo en entero.

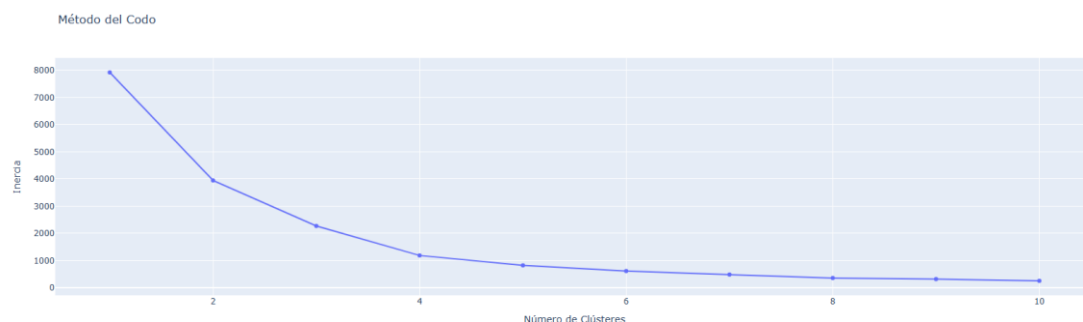
```
scaler = StandardScaler()
X_C = scaler.fit_transform(df_completo[['Basic_Demos-Age', 'Basic_Demos-Sex']])
```

Luego escalamos las variables sexo y edad por la diferencia en los rangos.

```
inertia = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='random', random_state=0)
    kmeans.fit(X_C)
    inertia.append(kmeans.inertia_)
```

Utilizamos algoritmo KMeans para realizar los clústeres y el método del codo para saber cuál es el número de clúster óptimo.

```
# Graficar el gráfico del codo con plotly
fig = px.line(x=range(1, 11), y=inertia, markers=True, title='Método del Codo')
fig.update_layout(xaxis_title='Número de Clústeres', yaxis_title='Inercia')
fig.show()
```



Podemos ver que el número de clúster óptimo es 3.

El propósito de realizar clústeres es poder crear grupo que tengan características en común y así poder dar tratamiento a los valores nulos y atípicos de acuerdo con cada uno de esos grupos. Se definieron algunas funciones para poder aplicar el tratamiento a estos datos.

```
kmeans = KMeans(n_clusters=3, init='random', random_state=0)
clusters = kmeans.fit_predict(X_C)
```

```
clusters
```

```
array([0, 0, 2, ..., 0, 1, 0], dtype=int32)
```

```
df_completo['Clusters'] = clusters
```

```
df_completo['Clusters'].value_counts()
```

```
Clusters
0    1689
2    1432
1     839
Name: count, dtype: int64
```

Vemos como se distribuyen los participantes en los diferentes clústeres.

```
df_cluster_0 = df_completo[df_completo['Clusters']== 0].copy()
df_cluster_1 = df_completo[df_completo['Clusters']== 1].copy()
df_cluster_2 = df_completo[df_completo['Clusters']== 2].copy()
```

Separamos los tres clústeres en dataframe diferentes.

A continuación, le aplicaremos algunas funciones que creamos para convertir variables que estaban en tipo float y object, sin embargo, son de tipo category.

```
convertir_columnas_categoricas_0(columnas_categoricas)
```

```
convertir_columnas_categoricas_1(columnas_categoricas)
```

```
convertir_columnas_categoricas_2(columnas_categoricas)
```

```
convertir_a_categoricas_0(columnas_str)
```

```
convertir_a_categoricas_1(columnas_str)
```

```
convertir_a_categoricas_2(columnas_str)
```

```
convertir_columnas_categoricas_1(columnas_str)
```

```
convertir_columnas_categoricas_2(columnas_str)
```

```
convertir_columnas_categoricas_0(columnas_str)
```

Luego de esto creamos una lista con los nombres de columnas categóricas que debemos de reemplazar los valores atípicos con la mediana y los valores nulos a cada clúster.

```
columnas_numericas = ["CGAS-CGAS_Score", "Physical-BMI", "Physical-Height", "Physical-Weight", "Physical-Waist_Circumference", "Physical-Diastolic_BP", "Physical-HeartRate",
    "Physical-Systolic_BP", "Fitness_Endurance-Max_Stage", "Fitness_Endurance-Time_Mins", "Fitness_Endurance-Time_Sec", "FGC-FGC_CU", "FGC-FGC_GSND",
    "FGC-FGC_GSD", "FGC-FGC_PU", "FGC-FGC_SRL", "FGC-FGC_SRR", "FGC-FGC_TL", "BIA-BIA_BMC", "BIA-BIA_BMI", "BIA-BIA_BMR", "BIA-BIA_DEE", "BIA-BIA_ECW",
    "BIA-BIA_FFM", "BIA-BIA_FFM1", "BIA-BIA_FMI", "BIA-BIA_Fat", "BIA-BIA_ICW", "BIA-BIA_LDM", "BIA-BIA_LST", "BIA-BIA_SMM", "BIA-BIA_TBW", "PAQ_A-PAQ_A_Total",
    "PAQ_C-PAQ_C_Total", "PCAT-PCAT_Total", "SDS-SDS_Total_Raw", "SDS-SDS_Total_T"]
```

```
reemplazar_atipicos_con_mediana_0(columnas_numericas)
```

```
reemplazar_atipicos_con_mediana_1(columnas_numericas)
```

```
reemplazar_atipicos_con_mediana_2(columnas_numericas)
```

```
sustitucion_valores_nulos_num_p(columnas_numericas, df_cluster_0)
```

```
sustitucion_valores_nulos_num_p(columnas_numericas, df_cluster_1)
```

```
sustitucion_valores_nulos_num_p(columnas_numericas, df_cluster_2)
```

Luego se aplicó el método info() al clúster 0 y vemos muestra de 20 de sus columnas para no abarcar mucho debido a la cantidad de columnas.

```
df_cluster_0.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1689 entries, 0 to 3959
Data columns (total 93 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         1689 non-null   object
1   Basic_Demos-Enroll_Season                 1689 non-null   category
2   Basic_Demos-Age                           1689 non-null   int64
3   Basic_Demos-Sex                           1689 non-null   category
4   CGAS-Season                               1689 non-null   category
5   CGAS-CGAS_Score                           1689 non-null   float64
6   Physical-Season                           1689 non-null   category
7   Physical-BMI                              1689 non-null   float64
8   Physical-Height                           1689 non-null   float64
9   Physical-Weight                           1689 non-null   float64
10  Physical-Waist_Circumference               1689 non-null   float64
11  Physical-Diastolic_BP                     1689 non-null   float64
12  Physical-HeartRate                        1689 non-null   float64
13  Physical-Systolic_BP                      1689 non-null   float64
14  Fitness_Endurance-Season                  1689 non-null   category
15  Fitness_Endurance-Max_Stage               1689 non-null   float64
16  Fitness_Endurance-Time_Mins               1689 non-null   float64
17  Fitness_Endurance-Time_Sec               1689 non-null   float64
18  FGC-Season                                1689 non-null   category
19  FGC-FGC_CU                                1689 non-null   float64
20  FGC-FGC_CU_Zone                           1689 non-null   category
```

```
def nueva_columna(df):
    df["Participo_Prueba"] = ["Si" if dato is not None and not pd.isna(dato) else "No" for dato in df["X"]]
    df["Participo_Prueba"] = df["Participo_Prueba"].astype("category")
```

```
# Creando la columna participo prueba para poder diferenciar los participantes
nueva_columna(df_cluster_0)
nueva_columna(df_cluster_1)
nueva_columna(df_cluster_2)
```

```
df_cluster_0["Participo_Prueba"].value_counts()
df_cluster_1["Participo_Prueba"].value_counts()
df_cluster_2["Participo_Prueba"].value_counts()
```

```
Participo_Prueba
No      1094
Si       338
Name: count, dtype: int64
```

En estas líneas de código se creó una función para crear una columna donde nos indique si participo o no en el grupo selecto para el estudio con mas detalles. Se crearon las columnas para cada clúster y como vemos esta es la distribución del clúster 2 donde nos dice que 338 de ese clúster participaron.

```
df_cluster_1.info()
```

```
81  sii                547 non-null    float64
82  X                  215 non-null    float64
83  Y                  215 non-null    float64
84  Z                  215 non-null    float64
85  enmo               215 non-null    float64
86  light              215 non-null    float64
87  battery_voltage    215 non-null    float64
88  time_of_day        215 non-null    float64
89  relative date PCIAT 215 non-null    float64
```

Luego de trabajar lo mencionado anteriormente, tuvimos que utilizar dos algoritmos KNNImputer y SimpleImputer para imputar los valores faltantes del grupo de columnas por clúster de la tabla inicial series_train.parquet. creamos dos funciones, las cuales están detalladas más arriba. Tuve que utilizar estos dos algoritmos debido a la limitante que tiene el primero, ya que imputar los valores faltantes de acuerdo con la distancia euclidiana y no pudieron imputarse todos ya que mucho no tenían vecinos cerca por decirlo así y a raíz de eso tuvimos que utilizar el otro para complementarlo.

```
columnas_a_imputar = ["sii", "X", "Y", "Z", "enmo", "light", "battery_voltage", "time_of_day", "relative_date_PCIAT"]
```

```
imputar_con_KNN(columnas_a_imputar , df_cluster_0)
imputar_con_KNN(columnas_a_imputar , df_cluster_1)
imputar_con_KNN(columnas_a_imputar , df_cluster_2)
```

```
df_cluster_0 = imputar_con_SI(columnas_a_imputar, df_cluster_0)
df_cluster_1 = imputar_con_SI(columnas_a_imputar, df_cluster_1)
df_cluster_2 = imputar_con_SI(columnas_a_imputar, df_cluster_2)
```

Como se pudo observar, se creo una lista con las columnas, luego se aplicaron las funciones con los algoritmos KNNImputer y SimpleImputer.

```
df_cluster_2.info()
```

```
80 PreInt_EduHx-computerinternet_hoursday 1432 non-null category
81 sii 1432 non-null float64
82 X 1432 non-null float64
83 Y 1432 non-null float64
84 Z 1432 non-null float64
85 enmo 1432 non-null float64
86 light 1432 non-null float64
87 battery_voltage 1432 non-null float64
88 time_of_day 1432 non-null float64
89 relative_date_PCIAT 1432 non-null float64
90 weekday 1432 non-null category
91 quarter 1432 non-null category
92 Clusters 1432 non-null int32
93 Participo_Prueba 1432 non-null category
dtypes: category(45), float64(46), int32(1), int64(1), object(1)
memory usage: 625.1+ KB
```

Como se puede en esta muestra del clúster 2, los valores de las columnas fueron imputados por los algoritmos mencionados y ahora podemos unificarlos para realizar el análisis exploratorio de los datos procesados y limpios.

```
print(df_cluster_0.shape)
print(df_cluster_1.shape)
print(df_cluster_2.shape)
```

```
(1689, 94)
(839, 94)
(1432, 94)
```

Aquí se puede ver la cantidad de filas y columnas por clúster. Estos datos están procesados y limpios. Ahora lo unificaremos para tener dataframe principal al cual le aplicaremos el análisis exploratorio.

Análisis Exploratorio de los datos

```
df = pd.concat([df_cluster_0, df_cluster_1, df_cluster_2], ignore_index = True)
```

```
df.duplicated().sum()
```

0

En estas líneas de código unificamos los tres clústeres y revisamos si tenían valores duplicados.

```
df.head()
```

```
[0]:
```

	id	Basic-Demos-Enroll_Season	Basic-Demos-Age	Basic-Demos-Sex	CGAS-Season	CGAS-Score	Physical-Season	Physical-BMI	Physical-Height	Physical-Weight	...	Z	enmo	light	battery_voltage	time_of_day	relative_date_PCIAI	weekday	quarter	Clusters	Participo_Pueba
0	00000ff9	Fall	5	0	Winter	51.0	Fall	16.877316	46.0	50.8	...	0.014190	0.02774	36.503995	3848.867383	5.571138e+12	58.302728	5	1	0	No
1	00005460	Summer	9	0	Spring	65.0	Fall	14.035590	48.0	46.0	...	0.014190	0.02774	36.503995	3848.867383	5.571138e+12	58.302728	5	1	0	No
2	00115b9f	Winter	9	0	Fall	71.0	Summer	18.292347	56.0	81.6	...	0.014190	0.02774	36.503995	3848.867383	5.571138e+12	58.302728	5	3	0	Si
3	0038ba98	Fall	10	0	Spring	65.0	Fall	19.660760	55.0	84.6	...	0.014190	0.02774	36.503995	3848.867383	5.571138e+12	58.302728	5	1	0	No
4	00abe655	Fall	11	0	Summer	66.0	Spring	16.871582	52.0	64.6	...	-0.128379	0.04656	44.981426	3880.435710	3.607885e+12	59.438255	5	1	0	No

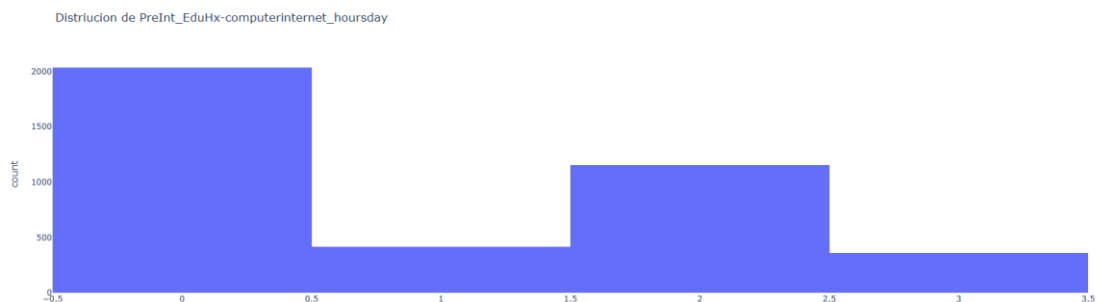
```
df.shape
```

(3960, 94)

En estas líneas anteriores podemos ver las primeras 5 filas del nuevo df y su forma, el cual contiene 94 columnas y 3960 filas .

Análisis Univariante

```
fig = px.histogram(df, x = "PreInt_EduHx-computerinternet_hoursday", title = "Distriucion de PreInt_EduHx-computerinternet_hoursday")  
fig.update_layout(  
    plot_bgcolor='white', # Fondo del área de trazado  
)  
fig.show()
```



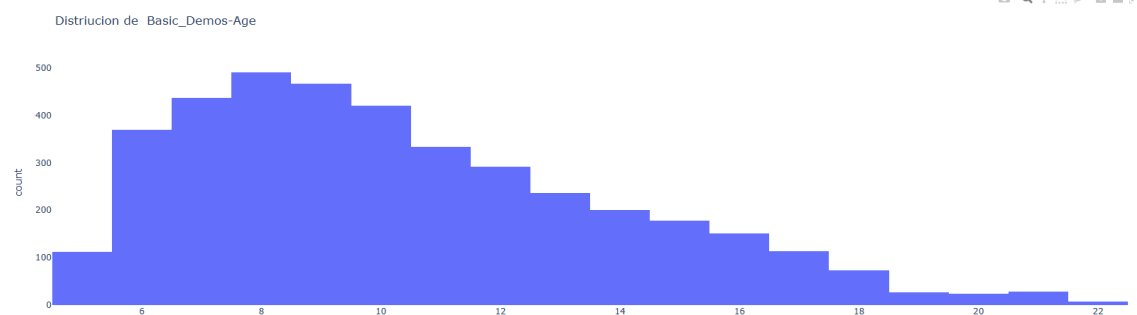
Como podemos observar de acuerdo con los datos:

- 51% de los participantes duran menos de una hora usando internet al día
- 10% de los participantes duran una hora usando internet al día

- 29% de los participantes duran dos horas usando internet por día
- 9% de los participantes duran más de tres horas al día usando el internet

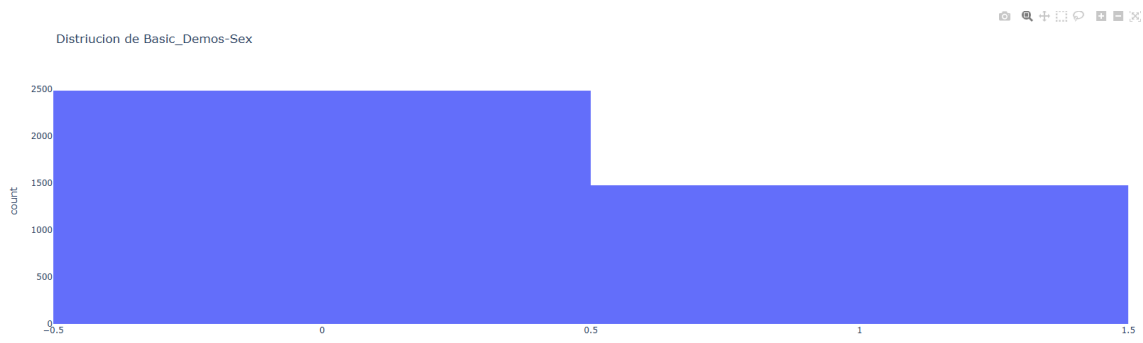
De acuerdo con estudios se argumentó que aquellas personas que pasan entre 8.5 horas a 21.5 por semana se considera uso problemático. Basando en esto se podría decir que el 38% de los participantes pueden tener uso problemático del internet.

```
fig = px.histogram(df, x = "Basic_Demos-Age", title = "Distriucion de Basic_Demos-Age" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



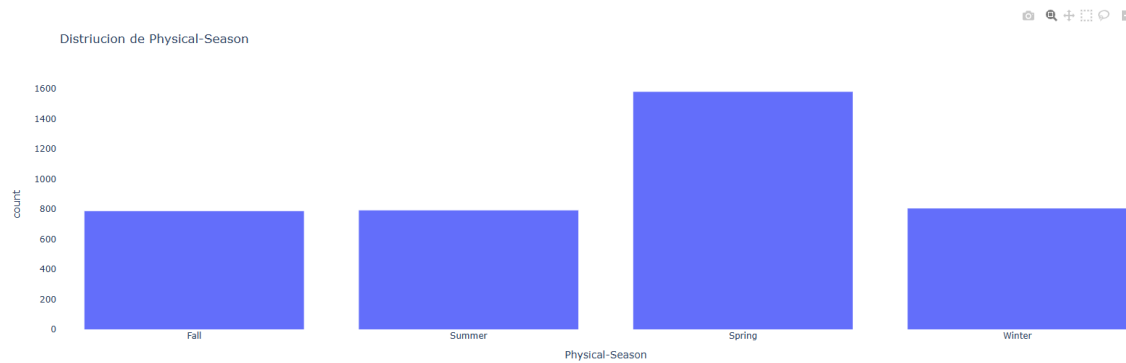
Aquí se puede observar que el 64% de los participantes están en la edad de la niñez y el 29% en la adolescencia.

```
fig = px.histogram(df, x = "Basic_Demos-Sex", title = "Distriucion de Basic_Demos-Sex" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



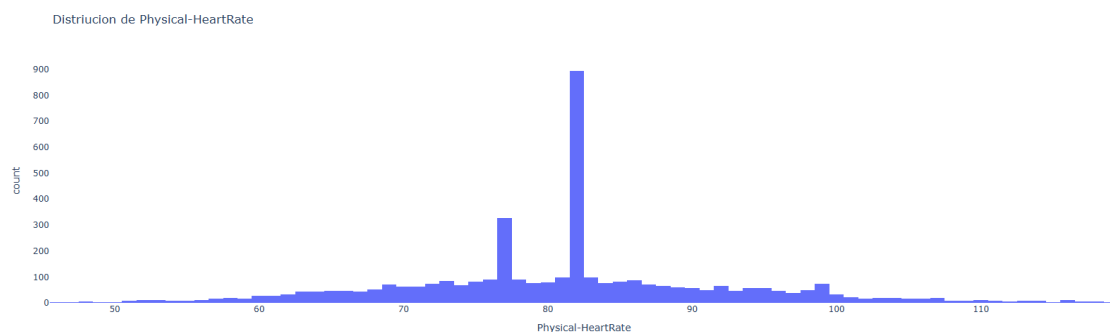
En este grafico vemos que el 63% de los participantes son de sexo masculino y el 37% femenino. En estudios previos se ha observado que las mujeres tienen a tener una mayor propensión al uso excesivo del internet.

```
fig = px.histogram(df, x = "Physical-Season", title = "Distriucion de Physical-Season" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



Vemos que el 40% de los participantes realizan las actividades físicas en primavera.

```
fig = px.histogram(df, x = "Physical-HeartRate", title = "Distriucion de Physical-HeartRate" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```

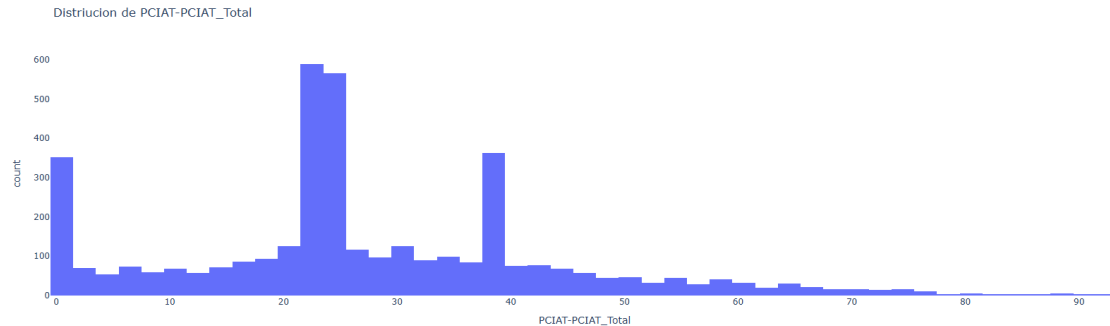


```
df\
.groupby(["Basic_Demos-Sex"], as_index = False)\
.agg(promedio_latido = ("Physical-HeartRate", "mean"),
     maximo_latidos = ("Physical-HeartRate", "max"))
```

	Basic_Demos-Sex	promedio_latido	maximo_latidos
0	0	80.856683	119.0
1	1	81.930894	118.0

En cuadro y grafico podemos observar el promedio de latidos del corazón por minutos está dentro del rango normal tanto del sexo masculino como femenino.

```
fig = px.histogram(df, x = "PCIAT-PCIAT_Total", title = "Distriucion de PCIAT-PCIAT_Total" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
print("Cantidad de participante con Nivel de uso severo:", len(df[df["PCIAT-PCIAT_Total"] >= 80]))
print("Cantidad de participante con Nivel de uso moderado:", len(df[(df["PCIAT-PCIAT_Total"] >= 50) & (df["PCIAT-PCIAT_Total"] <= 79)]))
print("Cantidad de participante con Nivel de uso leve:", len(df[(df["PCIAT-PCIAT_Total"] >= 31) & (df["PCIAT-PCIAT_Total"] <= 49)]))
print("Cantidad de participante con Nivel de uso adecuado:", len(df[(df["PCIAT-PCIAT_Total"] >= 0) & (df["PCIAT-PCIAT_Total"] <= 30)]))
```

```
Cantidad de participante con Nivel de uso severo: 25
Cantidad de participante con Nivel de uso moderado: 374
Cantidad de participante con Nivel de uso leve: 1022
Cantidad de participante con Nivel de uso adecuado: 2539
```

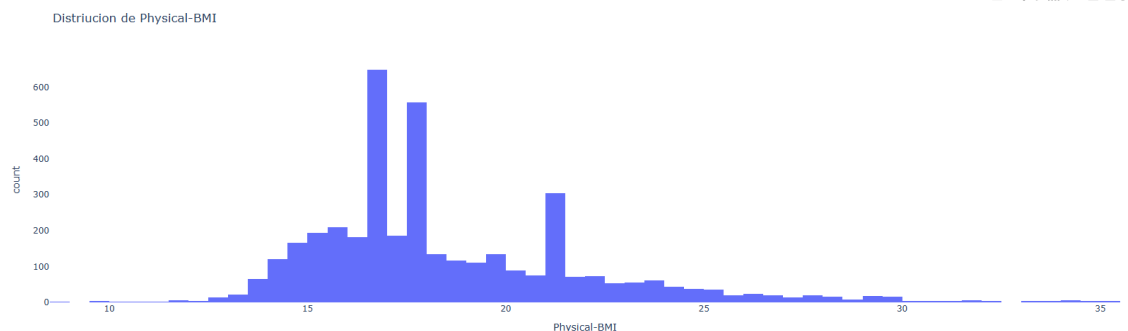
1% tiene un nivel de uso del internet severo.

10% tiene un nivel de uso moderado del internet

26% tiene un nivel de uso leve del internet.

64% tiene un nivel de uso dentro de lo normal.

```
fig = px.histogram(df, x = "Physical-BMI", title = "Distriucion de Physical-BMI")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
print("Cantidad de participante con insuficiencia de peso:", len(df[df["Physical-BMI"] < 18.5]))
print("Cantidad de participante con peso normal:", len(df[(df["Physical-BMI"] >= 18.5) & (df["Physical-BMI"] <= 24.9)]))
print("Cantidad de participante con sobrepeso:", len(df[(df["Physical-BMI"] >= 25) & (df["Physical-BMI"] <= 30)]))
print("Cantidad de participante con obesidad:", len(df[df["Physical-BMI"] > 30]))
```

Cantidad de participante con insuficiencia de peso: 2511
 Cantidad de participante con peso normal: 1219
 Cantidad de participante con sobrepeso: 186
 Cantidad de participante con obesidad: 35

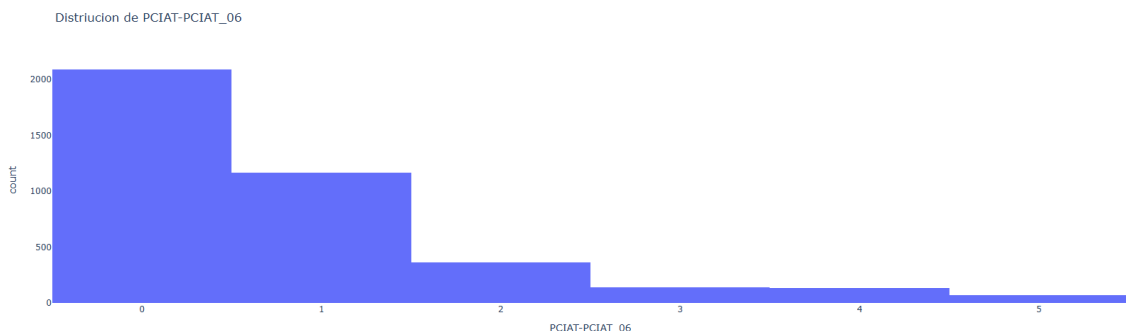
Cantidad de participante con insuficiencia de peso 63%.

Cantidad de participante con peso normal 31%

Cantidad de participante con sobrepeso 5%

Cantidad de participante con obesidad 1%

```
fig = px.histogram(df, x = "PCIAT-PCIAT_06", title = "Distriucion de PCIAT-PCIAT_06")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



Esta columna nos informa con qué frecuencia las calificaciones de los participantes se vio afectado por el uso del internet

EL 53% de los participantes dicen que sus calificaciones se ven afectadas.

El 29% de los participantes dicen que raras las veces su calificación se ven afectados

El 9% de los participantes dicen que ocasionalmente

EL 4% de los participantes dicen que Frecuentemente

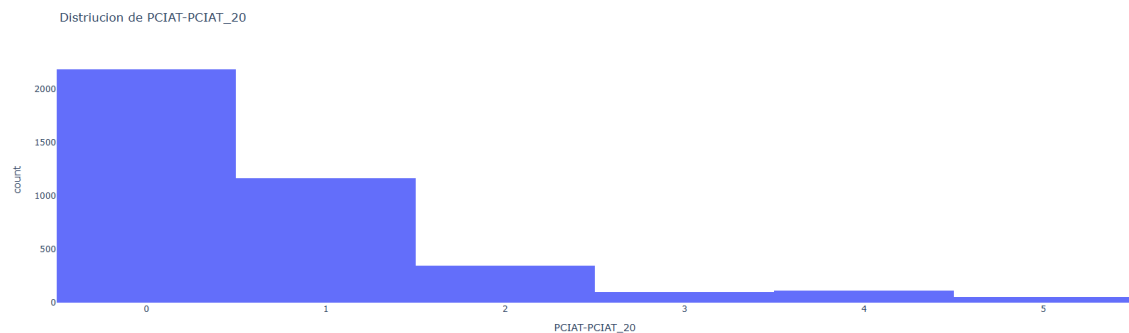
El 3% de los participantes dicen que a menudo

EL 2% de los participantes dicen que siempre

```
fig = px.histogram(df, x = "PCIAT-PCIAT_20", title = "Distriucion de PCIAT-PCIAT_20")

fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)

fig.show()
```



¿Con qué frecuencia su hijo se siente deprimido, de mal humor o nervioso cuando no está conectado y eso parece desaparecer cuando vuelve a conectarse?

EL 55% de los participantes dicen que no se ven afectadas.

El 29% de los participantes dicen que raras veces.

El 9% de los participantes dicen que ocasionalmente

EL 3% de los participantes dicen que Frecuentemente

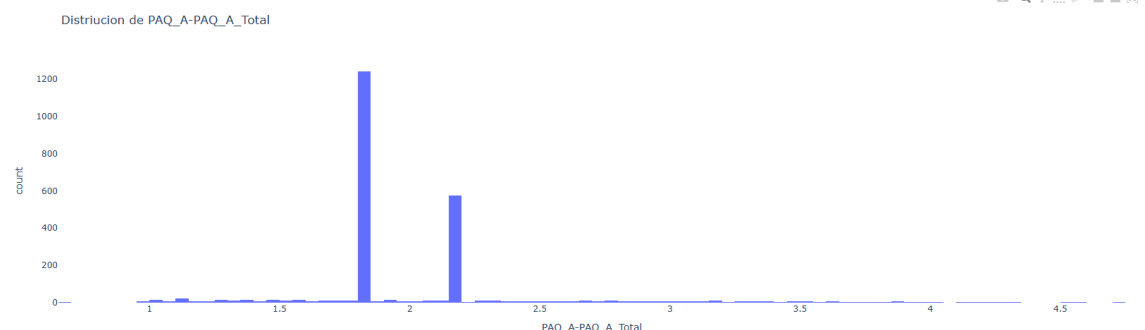
El 3% de los participantes dicen que a menudo

EL 1% de los participantes dicen que siempre

```
fig = px.histogram(df, x = "PAQ_A-PAQ_A_Total", title = "Distriucion de PAQ_A-PAQ_A_Total")

fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)

fig.show()
```



```
print("Cantidad de participante adolescentes bajo nivel de actividad fisica:", len(df[(df["PAQ_A-PAQ_A_Total"] >= 1) & (df["PAQ_A-PAQ_A_Total"] <= 1.9)]))
print("Cantidad de participante adolescentes nivel moderado de actividad fisica:", len(df[(df["PAQ_A-PAQ_A_Total"] >= 2) & (df["PAQ_A-PAQ_A_Total"] <= 2.9)]))
print("Cantidad de participante adolescentes alto nivel de actividad fisica:", len(df[(df["PAQ_A-PAQ_A_Total"] > 3)]))
```

```
Cantidad de participante adolescentes bajo nivel de actividad fisica: 1436
Cantidad de participante adolescentes nivel moderado de actividad fisica: 703
Cantidad de participante adolescentes alto nivel de actividad fisica: 91
```

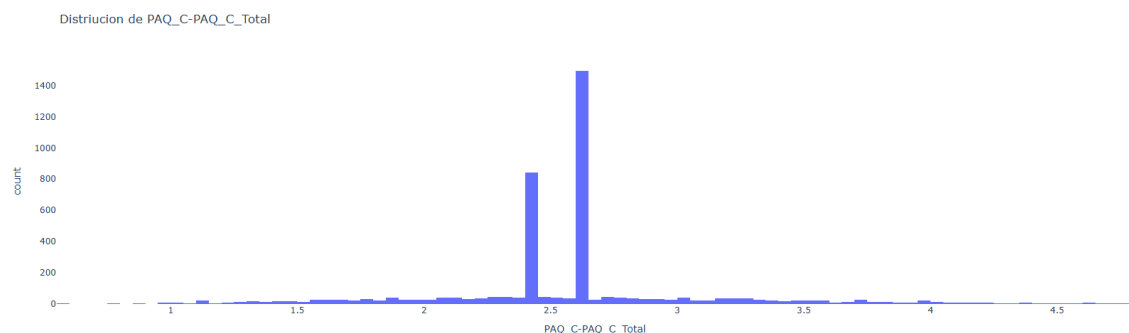
Esta métrica, PAQ_A-PAQ_A_Total, nos permite saber los niveles de actividades de los adolescentes la cual está muy relacionado con algunos aspectos de la salud. Sabemos a través de estudios y comentarios de expertos que a mayor nivel de actividad se puede tener mejor sistema cardiovascular, mayor fuerza muscular, mejor riesgo de obesidad. Adicional a esto también puede tener menor niveles de estrés, ansiedad y depresión.

El 64% de los adolescentes tuvieron un nivel de actividad física baja

El 31% de los adolescentes tuvieron un nivel de actividad física moderada

El 6% de los adolescentes tuvieron un nivel de actividad física alta

```
fig = px.histogram(df, x = "PAQ_C-PAQ_C_Total", title = "Distriucion de PAQ_C-PAQ_C_Total")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
print("Cantidad de participante adolescentes bajo nivel de actividad fisica:", len(df[(df["PAQ_C-PAQ_C_Total"] >= 1) & (df["PAQ_C-PAQ_C_Total"] <= 1.9)]))
print("Cantidad de participante adolescentes nivel moderado de actividad fisica:", len(df[(df["PAQ_C-PAQ_C_Total"] >= 2) & (df["PAQ_C-PAQ_C_Total"] <= 2.9)]))
print("Cantidad de participante adolescentes alto nivel de actividad fisica:", len(df[(df["PAQ_C-PAQ_C_Total"] > 3)]))
```

```
Cantidad de participante adolescentes bajo nivel de actividad fisica: 353
Cantidad de participante adolescentes nivel moderado de actividad fisica: 2958
Cantidad de participante adolescentes alto nivel de actividad fisica: 527
```

Esta medida, PAQ_C-PAQ_C_Total, nos permite conocer los niveles de actividades de los niños la cual está muy relacionado con algunos aspectos de la salud. Sabemos a través de estudios y comentarios de expertos que a mayor nivel de actividad se puede tener mejor sistema cardiovascular, mayor fuerza muscular, mejor riesgo de obesidad. Adicional a esto también puede tener menor niveles de estrés, ansiedad y depresión.

El 9% de los adolescentes tuvieron un nivel de actividad física baja

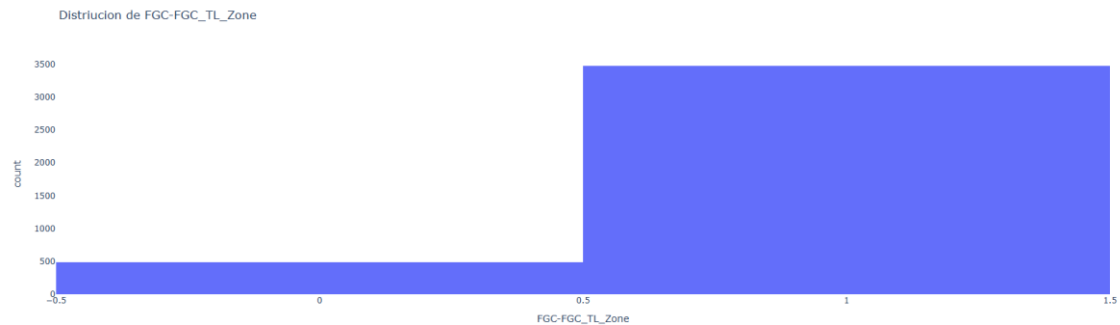
El 75% de los adolescentes tuvieron un nivel de actividad física moderada

El 16% de los adolescentes tuvieron un nivel de actividad física alta

```
fig = px.histogram(df, x = "FGC-FGC_TL_Zone", title = "Distriucion de FGC-FGC_TL_Zone")

fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)

fig.show()
```



Esta métrica está relacionada o hace referencia a la fuerza del tronco de los niños o adolescentes. Aquellos participantes que tengan un nivel de fuerza más considerable se pueden relacionar con un nivel mayor de actividad física y por ende menor sedentarismo por lo tanto menos riesgo de usas de manera problemática el internet.

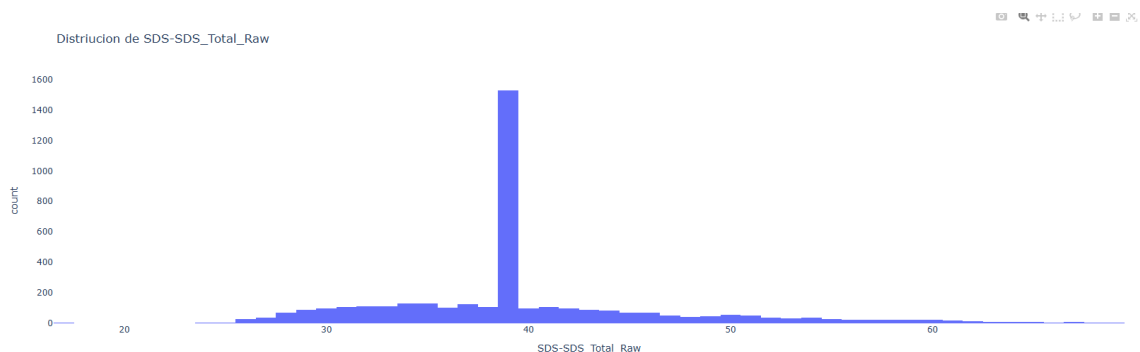
El 88% de los participantes tiene una fuerza del tronco adecuada, lo que podría sugerir unos niveles altos de actividad física.

El 12% de los participantes tienen menos fuerza en el tronco, lo que podría sugerir una baja o inadecuada actividad física.

```
fig = px.histogram(df, x = "SDS-SDS_Total_Raw", title = "Distriucion de SDS-SDS_Total_Raw")

fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)

fig.show()
```



```
print("Cantidad de participante sin perturbaciones en el sueño :", len(df[df["SDS-SDS_Total_Raw"] <= 29]))
print("Cantidad de participante con pocas perturbaciones en el de sueños, normales :", len(df[(df["SDS-SDS_Total_Raw"] >= 30) & (df["SDS-SDS_Total_Raw"] <= 50)]))
print("Cantidad de participante con moderada perturbaciones de sueños:", len(df[(df["SDS-SDS_Total_Raw"] > 50) & (df["SDS-SDS_Total_Raw"] <= 55)]))
print("Cantidad de participante con trastornos significativos de sueños:", len(df[(df["SDS-SDS_Total_Raw"] > 55) & (df["SDS-SDS_Total_Raw"] <= 60)]))
print("Cantidad de participante con graves tratosenos de sueños:", len(df[df["SDS-SDS_Total_Raw"] > 60]))
```

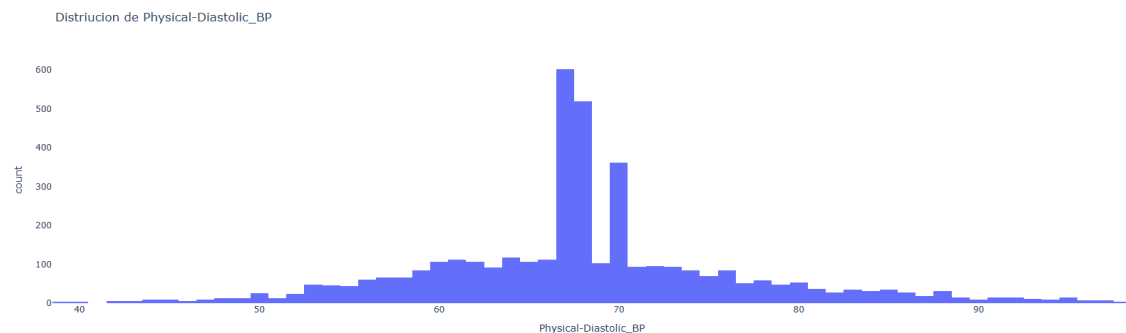
Cantidad de participante sin perturbaciones en el sueño : 232
 Cantidad de participante con pocas perturbaciones en el de sueños, normales : 3358
 Cantidad de participante con moderada perturbaciones de sueños: 182
 Cantidad de participante con trastornos significativos de sueños: 115
 Cantidad de participante con graves trastornos de sueños: 73

De acuerdo con estos datos, el 85% tiene un patrón de sueño normal, 5% tiene perturbaciones moderadas, 3% trastornos significativos de sueños y el 2% restantes graves trastornos.

Se podría decir que el 5% de los participantes podría requerir atención médica debido a sus trastornos.

```

fig = px.histogram(df, x = "Physical-Diastolic_BP", title = "Distriucion de Physical-Diastolic_BP" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
  
```



```

print("Cantidad de participante tienen la presión diastólica muy baja :", len(df[df["Physical-Diastolic_BP"] < 60]))
print("Cantidad de participante tienen la presión diastólica normal :", len(df[(df["Physical-Diastolic_BP"] >= 60) & (df["Physical-Diastolic_BP"] <= 80)]))
print("Cantidad de participante tienen la presión diastólica alta :", len(df[df["Physical-Diastolic_BP"] > 80]))
  
```

Cantidad de participante tienen la presión sistólica muy baja : 39
 Cantidad de participante tienen la presión sistólica normal : 2796
 Cantidad de participante tienen la presión sistólica elevada : 720
 Cantidad de participante estan en etapa de hipertensión : 373

De acuerdo con estos datos el 71% de los participantes tienen la presión arterial sistólica normal.

1% la tiene muy baja.

18% la tiene elevada.

10% está en etapa de hipertensión.

Estos resultados se deben de considerar los niveles de actividad que estaban realizando.

```
# Estoy creando una nueva cloumna tipo object para utilizarla en el análisis bivariante.
def clasificar(valor):
    if 0.0 <= valor <= 30.0:
        return "Ninguna"
    elif 31.0 <= valor <= 49.0:
        return "leve"
    elif 50.0 <= valor <= 79.0:
        return "moderada"
    else:
        return "severo"

df["sii_m"] = df["PCIAT-PCIAT_Total"].apply(clasificar)
```

En este código estamos la columna sii_m que no es más que la variable sii en tipo de dato category. Esto lo utilizamos para ver discriminar el comportamiento de otras variables categóricas influenciada por esta.

```
# Revisando las distribucion de los resutlados
df["sii_m"].value_counts()
```

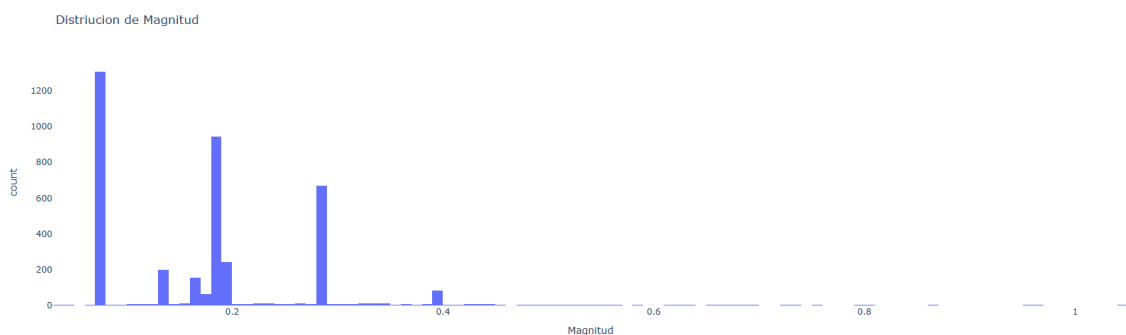
```
sii_m
Ninguna      2539
leve         1022
moderada      374
severo        25
Name: count, dtype: int64
```

```
# Estas columnas que reprentan la aceleración, se deben de elevar al cuadrado y srealizar una suma con ellas para poder analizarlas
df['Magnitud'] = np.sqrt(df['X']**2 + df['Y']**2 + df['Z']**2)
```

```
fig = px.histogram(df, x = "Magnitud", title = "Distriucion de Magnitud" )

fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)

fig.show()
```



```
# Estoy creando una columna estado movimiento para saber si los participantes estan activo o inactivos

def clasificar_estado(valor):
    if 0.0 <= valor < 0.01:
        return "inactivo"
    if 0.01 <= valor <= 0.04:
        return "baja actividad"
    else:
        return "actividad moderada"

df["Estado_movimiento"] = df["enmo"].apply(clasificar_estado)
```

```
# Revisando la distribución de los resultados
df["Estado_movimiento"].value_counts()
```

```
Estado_movimiento
baja actividad      3567
actividad moderada   354
inactivo            39
Name: count, dtype: int64
```

```
# Se esta tratando a columna time_of_day para poder calcular los periodos de actividad mañana, tarde o noche.
df['time_of_day'] = pd.to_datetime(df['time_of_day'], unit = 'ns')
fecha_actual = datetime.now()
epoch_date = datetime(1970, 1, 1)
df['time_of_day'] = df['time_of_day'] + (fecha_actual - epoch_date)
```

```
# Luego de tratar la columna esoty creando la columna horario.
def clasificar_horario(hora):
    hora_dt = pd.to_datetime(hora, format='%H:%M:%S.%f').time()
    if hora_dt >= pd.to_datetime('06:00:00').time() and hora_dt < pd.to_datetime('12:00:00').time():
        return 'Mañana'
    elif hora_dt >= pd.to_datetime('12:00:00').time() and hora_dt < pd.to_datetime('18:00:00').time():
        return 'Tarde'
    else:
        return 'Noche'

df['Horario'] = df['time_of_day'].apply(clasificar_horario)
```

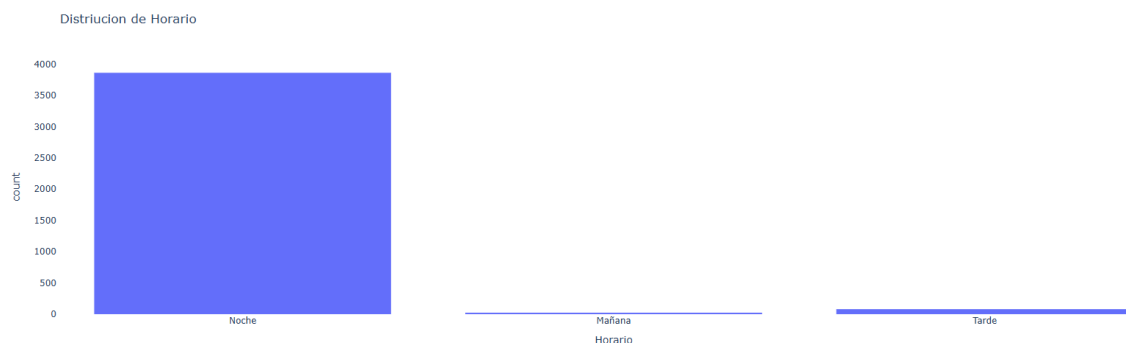
```
# Revision de la distribución de los valores de la columna horario.
df['Horario'].value_counts()
```

```
Horario
Noche      3862
Tarde       80
Mañana      18
Name: count, dtype: int64
```

```
fig = px.histogram(df, x = "Horario", title = "Distriucion de Horario")

fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)

fig.show()
```



```
df["Horario"].value_counts()
```

```

Horario
Noche      3862
Tarde       80
Mañana      18
Name: count, dtype: int64

```

Aquí podemos ver que en el horario de la tarde es donde más registraron actividad, como era de esperarse.

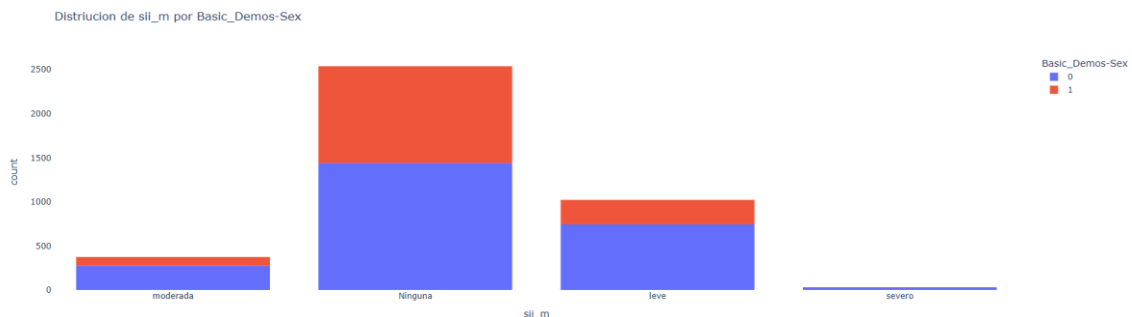
El 79% de las actividades se realizan en la tarde, 20% en la noche y 1% en la mañana. Esto se puede deber a que en ese horario estén aun durmiendo.

Análisis Bivariante

```

fig = px.histogram(df, x = "sii_m", title = "Distriucion de sii_m por Basic_Demos-Sex", color = "Basic_Demos-Sex")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()

```



Como podemos ver en el gráfico, los participantes de sexo masculino tienen un comportamiento más adictivo con el internet que las de sexo femenino. Se ve claramente el comportamiento discriminatorio. Dado estos datos se puede considerar esta variable dentro del modelo. Como se puede en el nivel dependencia moderada, el 74% de esa categoría son de sexo masculino y en la categoría leve un 73%.

```

tabla_cruzada_1 = pd.crosstab(df['sii_m'], df['Basic_Demos-Sex'], margins = True)
tabla_cruzada_1

```

Basic_Demos-Sex	0	1	All
sii_m			
Ninguna	1440	1099	2539
leve	744	278	1022
moderada	276	98	374
severo	24	1	25
All	2484	1476	3960

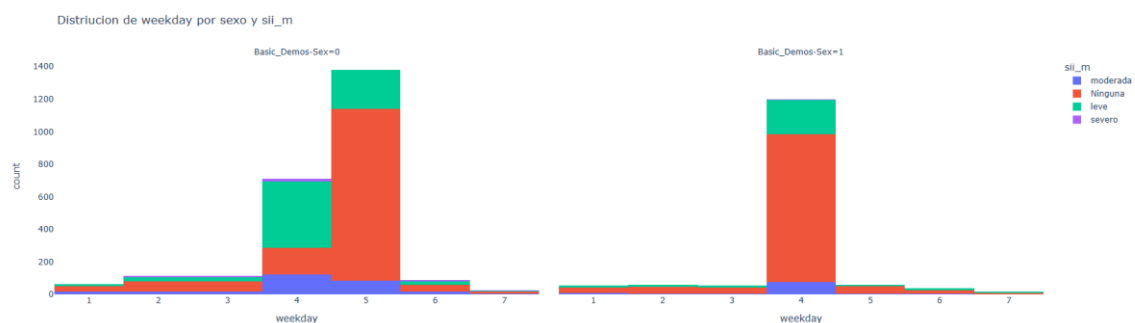
```
chi2, p, dof, expected = chi2_contingency(tabla_cruzada_1)
print("\nChi-cuadrado:")
print(f"Chi2: {chi2}, p-valor: {p}")
```

Chi-cuadrado:

Chi2: 115.02711698029069, p-valor: 3.517077128491876e-21

Dado el valor arrojado por el p-valor, al ser tan pequeño, que existe evidencia estadísticamente significativa de que tenemos una relación de dependencia entre estas dos variables.

```
fig = px.histogram(df, x = "weekday", title = "Distriucion de weekday por sexo y sii_m", color = "sii_m", facet_col = "Basic_Demos-Sex")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



Aquí por podemos ver de una forma más clara que el comportamiento adictivo se muestra más en los participantes del sexo masculino y los jueves tanto, para el sexo masculino como femenino presentan una conducta adictiva leve. Cabe destacar que el comportamiento no adictivo predomina en este grupo participantes.

```
tabla_cruzada_2 = pd.crosstab(df['sii_m'], df['weekday'], margins = True)
tabla_cruzada_2
```

weekday	1	2	3	4	5	6	7	All
sii_m								
Ninguna	71	108	100	1072	1100	65	23	2539
leve	26	39	40	618	251	38	10	1022
moderada	22	20	21	199	89	18	5	374
severo	0	1	3	18	0	2	1	25
All	119	168	164	1907	1440	123	39	3960

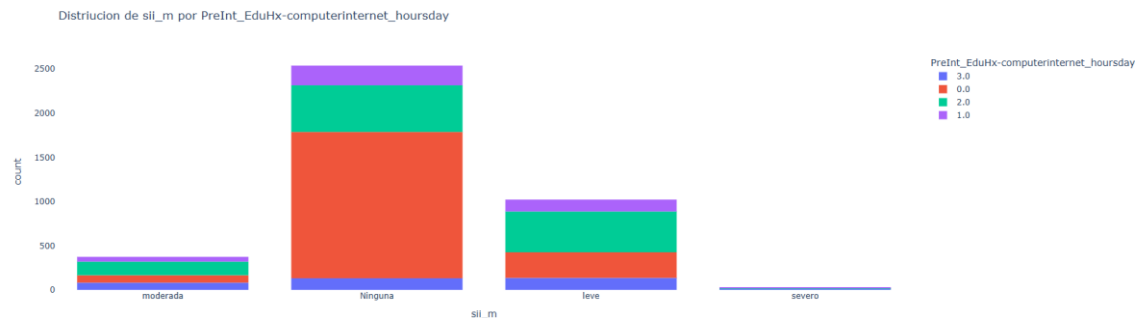
```
chi2, p, dof, expected = chi2_contingency(tabla_cruzada_2)
print("\nChi-cuadrado:")
print(f"Chi2: {chi2}, p-valor: {p}")
```

Chi-cuadrado:

Chi2: 185.78194273767198, p-valor: 3.250540888308112e-25

De acuerdo con el resultado arrojado del p-valor, al ser tan pequeño, existe evidencia estadísticamente significativa de que existe una asociación entre estas dos variables.

```
fig = px.histogram(df, x = "sii_m", title = "Distriucion de sii_m por PreInt_EduHx-computerinternet_hoursday", color = "PreInt_EduHx-computerinternet_hoursday")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
tabla_cruzada_3 = pd.crosstab(df['sii_m'], df['PreInt_EduHx-computerinternet_hoursday'], margins = True)
tabla_cruzada_3
```

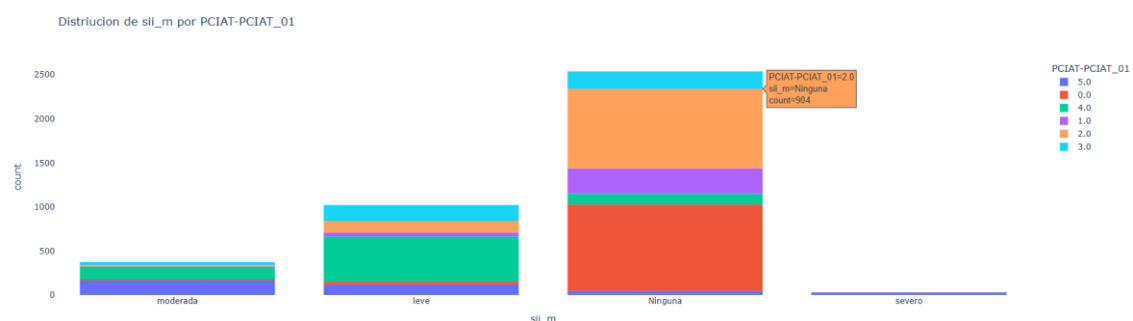
PreInt_EduHx-computerinternet_hoursday	0.0	1.0	2.0	3.0	All
sii_m					
Ninguna	1656	223	528	132	2539
leve	290	134	462	136	1022
moderada	84	54	154	82	374
severo	3	2	10	10	25
All	2033	413	1154	360	3960

```
chi2, p, dof, expected = chi2_contingency(tabla_cruzada_3)
print("\nChi-cuadrado:")
print(f"Chi2: {chi2}, p-valor: {p}")
```

Chi-cuadrado:
Chi2: 615.2641104393574, p-valor: 1.3208376076761666e-120

En el grafico no se podía ver bien si había una relación entre estas dos variables, sin embargo, con este valor arrojado de p-valoré vemos existe una evidencia estadísticamente significativa que existe una asociación entre estas variables.

```
fig = px.histogram(df, x = "sii_m", title = "Distriucion de sii_m por PCIAT-PCIAT_01", color = "PCIAT-PCIAT_01")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
tabla_cruzada_4 = pd.crosstab(df['sii_m'], df['PCIAT-PCIAT_01'], margins = True)
tabla_cruzada_4
```

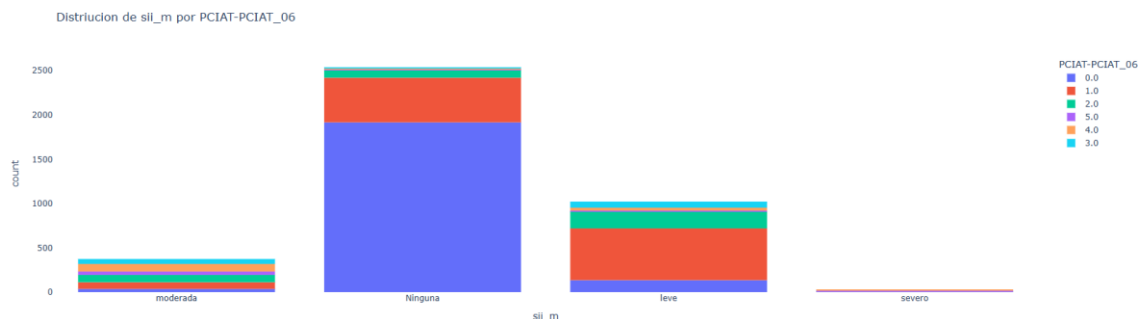
PCIAT-PCIAT_01	0.0	1.0	2.0	3.0	4.0	5.0	All
sii_m							
Ninguna	984	287	904	198	121	45	2539
leve	37	46	128	184	510	117	1022
moderada	6	5	18	36	139	170	374
severo	0	1	0	0	4	20	25
All	1027	339	1050	418	774	352	3960

```
chi2, p, dof, expected = chi2_contingency(tabla_cruzada_4)
print("\nChi-cuadrado:")
print(f"Chi2: {chi2}, p-valor: {p}")
```

Chi-cuadrado:
Chi2: 2499.116788778829, p-valor: 0.0

De acuerdo con estos resultados, existe evidencia estadísticamente significativa de que estas variables están relacionadas entre, esto mide el nivel de desobediencia y la conducta adictiva hacia el internet. Como podemos ver en el cuadro a mayor frecuencia se desobediencia más va mostrando una conducta adictiva mayor.

```
fig = px.histogram(df, x = "sii_m", title = "Distriucion de sii_m por PCIAT-PCIAT_06", color = "PCIAT-PCIAT_06")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
tabla_cruzada_5 = pd.crosstab(df['sii_m'], df['PCIAT-PCIAT_06'], margins = True)
tabla_cruzada_5
```

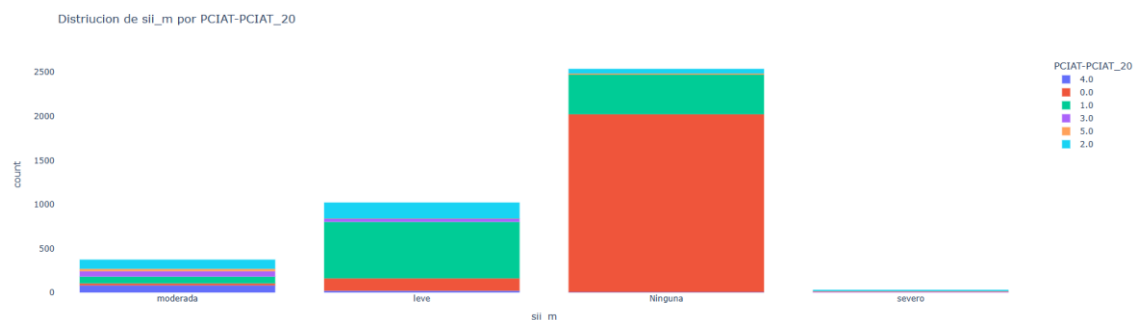
PCIAT-PCIAT_06	0.0	1.0	2.0	3.0	4.0	5.0	All
sii_m							
Ninguna	1917	504	87	16	9	6	2539
leve	135	585	192	69	33	8	1022
moderada	36	76	82	56	85	39	374
severo	0	0	1	0	7	17	25
All	2088	1165	362	141	134	70	3960

```
chi2, p, dof, expected = chi2_contingency(tabla_cruzada_5)
print("\nChi-cuadrado:")
print(f"Chi2: {chi2}, p-valor: {p}")
```

Chi-cuadrado:
Chi2: 2899.304816382496, p-valor: 0.0

De acuerdo con estos resultados, existe evidencia estadísticamente significativa de que estas variables están relacionadas entre, esto mide como se ve afectado su rendimiento académico y la conducta adictiva hacia el internet.

```
fig = px.histogram(df, x = "sii_m", title = "Distriucion de sii_m por PCIAT-PCIAT_20", color = "PCIAT-PCIAT_20")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



```
tabla_cruzada_6 = pd.crosstab(df['sii_m'], df['PCIAT-PCIAT_20'], margins = True)
tabla_cruzada_6
```

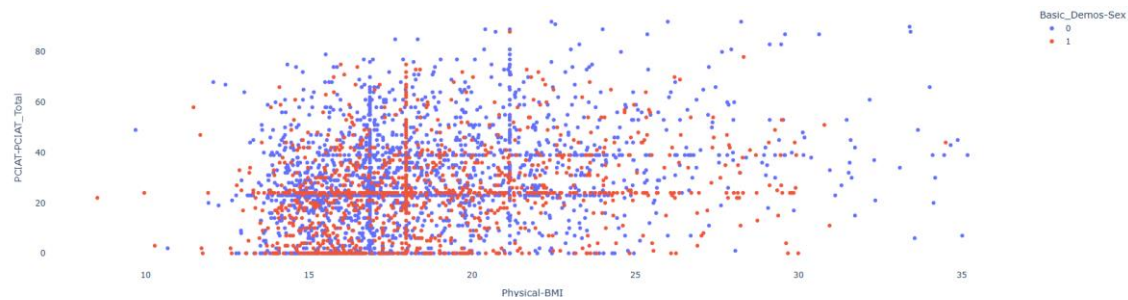
PCIAT-PCIAT_20	0.0	1.0	2.0	3.0	4.0	5.0	All
sii_m							
Ninguna	2018	450	54	3	5	9	2539
leve	141	641	182	36	20	2	1022
moderada	23	77	105	61	81	27	374
severo	0	0	2	2	7	14	25
All	2182	1168	343	102	113	52	3960

```
chi2, p, dof, expected = chi2_contingency(tabla_cruzada_6)
print("\nChi-cuadrado:")
print(f"Chi2: {chi2}, p-valor: {p}")
```

Chi-cuadrado:
Chi2: 3293.155728164593, p-valor: 0.0

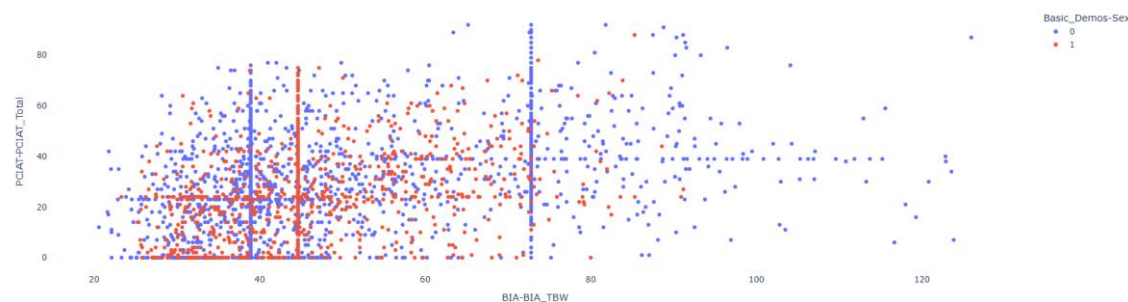
De acuerdo con estos resultados, existe evidencia estadísticamente significativa de que estas variables están relacionadas entre sí, esto mide el estado del ánimo y humor cuando no está conectado y la conducta adictiva hacia el internet.

```
fig = px.scatter(df, x = "Physical-BMI", y = "PCIAT-PCIAT_Total", color = "Basic-Demos-Sex" )
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



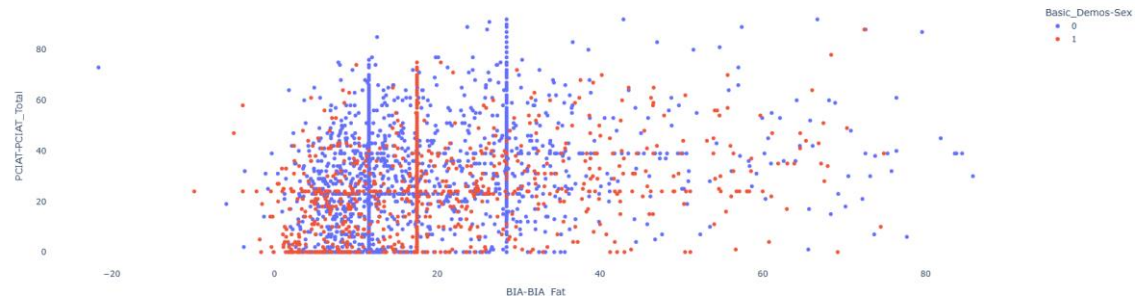
En este grafico se puede ver a que mayor índice masa corporal mayor puntuación que nos indica la conducta adictiva de los participantes. Explica que las personas con obesidad o sobrepeso son más propensas a este tipo de conducta viciosa. Vemos que tanto los hombres como las mujeres tienden a relación.

```
fig = px.scatter(df, x = "BIA-BIA_TBW", y = "PCIAT-PCIAT_Total", color = "Basic_Demos-Sex")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



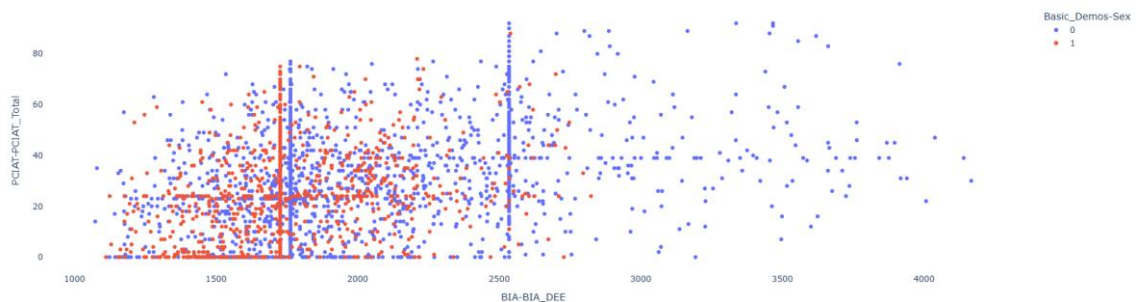
Aquí podemos ver la relación positiva sobre el índice PCIAT_Total y el total de agua en el cuerpo. Esto resulta evidente ya que las dos principales formas en la cual el agua del cuerpo se consume son: sudor y orina. Esta grafica nos dice que a mayor nivel agua en el cuerpo mayor en la puntuación la cual tiene sentido, ya que si tienes una vida sedentaria no vas a sudar y por no consumirás el agua del cuerpo. Y además nos confirma lo que hemos estado viendo en los gráficos, los hombres tienen más agua en el cuerpo que las mujeres lo que nos confirma que son más sedentarios en estos participantes.

```
fig = px.scatter(df, x = "BIA-BIA_Fat", y = "PCIAT-PCIAT_Total", color = "Basic_Demos-Sex")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



Aquí podemos ver la relación positiva sobre el índice PCIAT_Total y el total de grasa en el cuerpo. Esto resulta evidente ya que a mayor grasa menos actividad física. Esta grafica nos dice que a mayor nivel grasa en el cuerpo mayor en la puntuación la cual tiene sentido, ya que si tienes una vida sedentaria no vas a quemar y por no consumirás la grasa del cuerpo. Aquí vemos una distribución más equitativa entre el sexo masculino y femenino.

```
fig = px.scatter(df, x = "BIA-BIA_DEE", y = "PCIAT-PCIAT_Total", color = "Basic_Demos-Sex")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



De acuerdo con esta grafica se podría pensar que existe una relación positiva entre el gasto diario energético y el índice PCIAT, sin embargo, habría que confirmarlo con una correlación de Pearson para confirmarlo. A pesar de que parezca que no tiene sentido, debemos tener pendiente que existen deportes electrónicos y juegos que podrían confirmar esta relación.

```
df_int = df[(df["CGAS-CGAS_Score"] <= 60) & (df["Basic_Demos-Age"] < 18)]
len(df_int)
```

854

```
df_int\
.groupby(["sii_m"], as_index = False)\
.agg(promedio_edad = ("Basic_Demos-Age", "mean"),
     maximo_edad = ("Basic_Demos-Age", "max"),
     promedio_indice = ("PCIAT-PCIAT_Total", "mean"),
     cantidad = ("id", "size"))
```

	sii_m	promedio_edad	maximo_edad	promedio_indice	cantidad
0	Ninguna	8.770961	17	13.842536	489
1	leve	10.991803	17	39.213115	244
2	moderada	11.666667	17	59.877193	114
3	severo	14.714286	17	87.142857	7

La variable CGAS-CGAS_Score es la escala de evaluación global para niños que mide su salud mental y vemos en grafico un significativo de participantes que están del rango, 1 al 60, con algún tipo de problemas, para ser más exacto el 22% de los participantes. Realicé un cuadro donde podría ver el índice PCIAT, la edad promedio y máxima para tener un cuadro más exacto de este grupo y vemos que medida que tienen mayor edad tiene el índice que refleja la adicción más alto, ósea que tiene un comportamiento con tendencia a problemas con el internet, se podría decir el 46% de ellos tiene un síntoma leve.

```
# Creamos un subconjunto de dato para ver si encuentre algun patron con aquellos participante que tiene el indice de deterioro en categoria de severo.
df_PCIAT = df[df["PCIAT-PCIAT_Total"] >= 80]
```

```
df_PCIAT\
.groupby(["sii_m", "Basic-Demos-Sex"], as_index = False)\
.agg(promedio_edad = ("Basic-Demos-Age", "mean"),
     maximo_edad = ("Basic-Demos-Age", "max"),
     promedio_indice = ("PCIAT-PCIAT_Total", "mean"),
     cantidad = ("id", "size"))
```

	sii_m	Basic-Demos-Sex	promedio_edad	maximo_edad	promedio_indice	cantidad
0	severo	0	14.666667	19	86.0	24
1	severo	1	21.000000	21	88.0	1

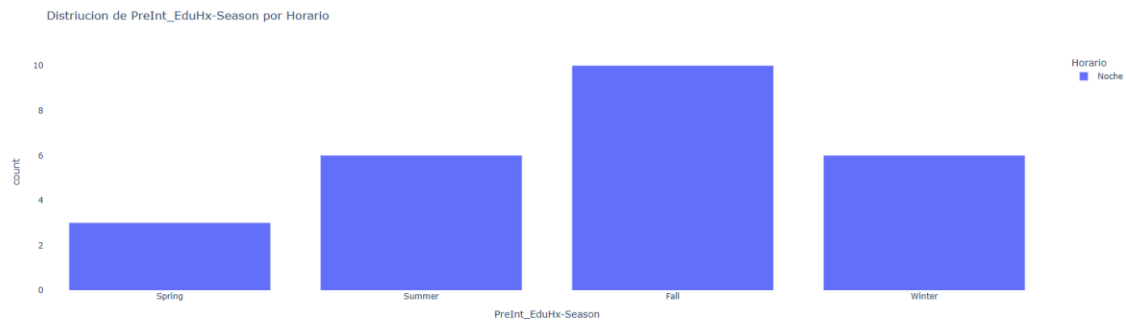
```
df_PCIAT\
.groupby(["sii_m", "PreInt_EduHx-computerinternet_hoursday"], as_index = False)\
.agg(promedio_IMC = ("Physical-BMI", "mean"),
     promedio_lc = ("Physical-HeartRate", "mean"),
     promedio_indice = ("CGAS-CGAS_Score", "mean"),
     cantidad = ("id", "size"))
```

	sii_m	PreInt_EduHx-computerinternet_hoursday	promedio_IMC	promedio_lc	promedio_indice	cantidad
0	severo	0.0	25.445824	70.0	65.333333	3
1	severo	1.0	20.039292	79.5	63.000000	2
2	severo	2.0	23.823974	75.1	58.400000	10
3	severo	3.0	26.700908	79.9	60.600000	10

```
df_PCIAT\
.groupby(["sii_m", "Basic-Demos-Sex"], as_index = False)\
.agg(promedio_IMC = ("Physical-BMI", "mean"),
     promedio_lc = ("Physical-HeartRate", "mean"),
     promedio_indice = ("CGAS-CGAS_Score", "mean"),
     cantidad = ("id", "size"))
```

	sii_m	Basic-Demos-Sex	promedio_IMC	promedio_lc	promedio_indice	cantidad
0	severo	0	25.021134	77.083333	61.333333	24
1	severo	1	21.157667	69.000000	40.000000	1

```
fig = px.histogram(df_PCIAT, x = "PreInt_EduHx-Season", title = "Distriucion de PreInt_EduHx-Season por Horario", color = "Horario")
fig.update_layout(
    plot_bgcolor='white', # Fondo del área de trazado
)
fig.show()
```



Aquí podemos ver en este grupo donde su índice es severo que el 96% son masculinos y tiene un promedio de edad de 14 años. En promedio todos están en sobrepeso de acuerdo con su índice de masa muscular. también tiene un índice de escala de evaluación global infantil están dentro de lo rangos en los tienen alguna dificultad en un área específica, pero en general funciona bastante bien en cuanto a los aspectos psicológicos y sociales. EL 76% de ellos dura más 2 horas usando el internet. También vemos en el grafico que la mayoría acostumar desde la mañana en el otoño.

Codificaciones variables categóricas

```
# Seleccionamos las columnas tipos object para codificarlas.
columnas_object = df.select_dtypes(include = "object")
```

```
# Estamos codificando
le = LabelEncoder()

for col in columnas_object:
    df[col] = le.fit_transform(df[col])
```

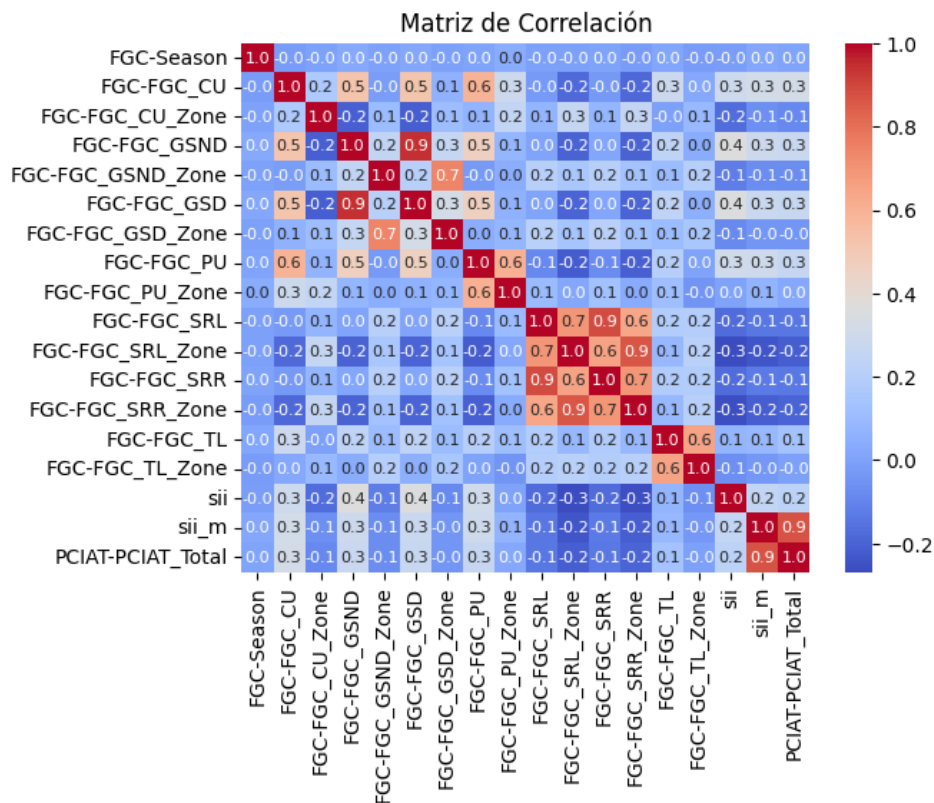
```
# Creamos una lista de la columnas tipo category para luego filtrar aquellas que contengan la palabra Season para aplicar codificarla.
lista_category = df.select_dtypes(include= 'category')
for data in lista_category:
    if "Season" in data:
        df[data] = le.fit_transform(df[data])
```

```
# Creamos una nueva lista con las columnas category que no incluyen en su nombre "Season", ya que estas solamente necesitaban convertir sus valores a int.
lista_category_2 = df.select_dtypes(include= 'category')
for data in lista_category_2:
    df[data] = df[data].cat.codes
```

Análisis de correlación

```
grupo_1 = ["Basic-Demos-Enroll_Season", "Basic-Demos-Age", "Basic-Demos-Sex", "CGAS-Season", "CGAS-CGAS_Score", "Physical-Season", "Physical-BMI", "Physical-Height", "Physical-Weight",
"Physical-Waist_Circumference", "Physical-Diastolic_BP", "Physical-HeartRate", "Physical-Systolic_BP", "Fitness_Endurance-Season", "Fitness_Endurance-Max_Stage",
"Fitness_Endurance-Time_Mins", "Fitness_Endurance-Time_Sec", "sii", "sii_m", "PCIAT-PCIAT_Total"]
```

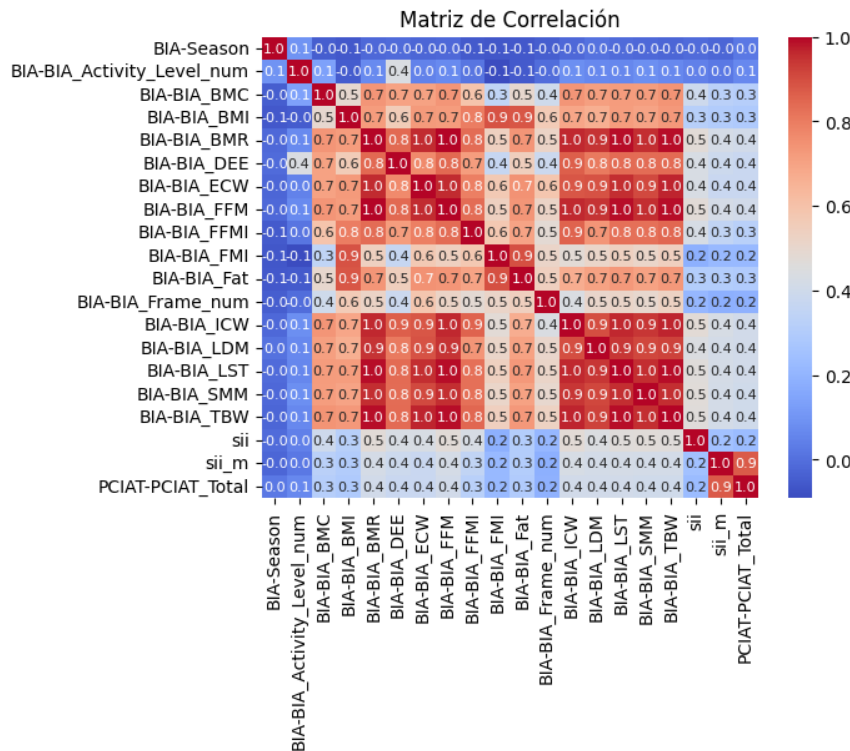
```
sns.heatmap(df[grupo_1].corr(), annot = True, cmap='coolwarm', fmt='.1f', annot_kws={"size": 8})
plt.title('Matriz de Correlación')
plt.show()
```

En este segundo grupo fuerza de agarre de la mano dominante, mano no dominante y acurrucarse total las variables que tiene el coeficiente más alto con el índice de deterioro de gravedad.

```
grupo_3 = ["BIA-Season", "BIA-BIA_Activity_Level_num", "BIA-BIA_BMC", "BIA-BIA_BMI", "BIA-BIA_BMR", "BIA-BIA_DEE", "BIA-BIA_ECW", "BIA-BIA_FFW", "BIA-BIA_FFMI", "BIA-BIA_FMI", "BIA-BIA_Fat",
           "BIA-BIA_Frame_num", "BIA-BIA_ICW", "BIA-BIA_LDM", "BIA-BIA_LST", "BIA-BIA_SMM", "BIA-BIA_TBW", "sii", "sii_m", "PCIAT-PCIAT_Total"]

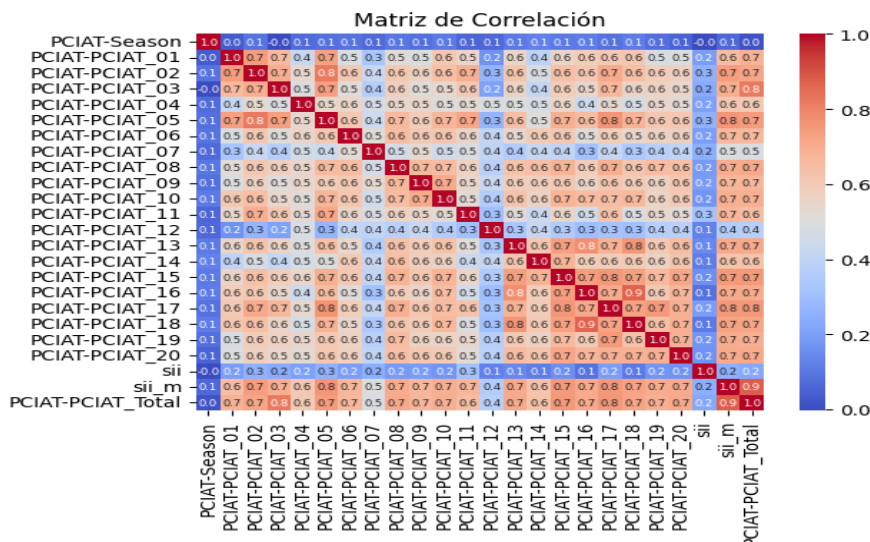
sns.heatmap(df[grupo_3].corr(), annot = True, cmap='coolwarm', fmt='.1f', annot_kws={"size": 8})
plt.title('Matriz de Correlación')
plt.show()
```



Vemos en este grupo que las variables con el coeficiente de correlación más alto son índice masa corporal, consumo diario energético, total de agua en el cuerpo, todos alrededor de un 4.

```
grupo_4 = ["PCIAT-Season", "PCIAT-PCIAT_01", "PCIAT-PCIAT_02", "PCIAT-PCIAT_03", "PCIAT-PCIAT_04", "PCIAT-PCIAT_05", "PCIAT-PCIAT_06", "PCIAT-PCIAT_07", "PCIAT-PCIAT_08", "PCIAT-PCIAT_09", "PCIAT-PCIAT_10", "PCIAT-PCIAT_11", "PCIAT-PCIAT_12", "PCIAT-PCIAT_13", "PCIAT-PCIAT_14", "PCIAT-PCIAT_15", "PCIAT-PCIAT_16", "PCIAT-PCIAT_17", "PCIAT-PCIAT_18", "PCIAT-PCIAT_19", "PCIAT-PCIAT_20", "sii", "sii_m", "PCIAT-PCIAT_Total"]
```

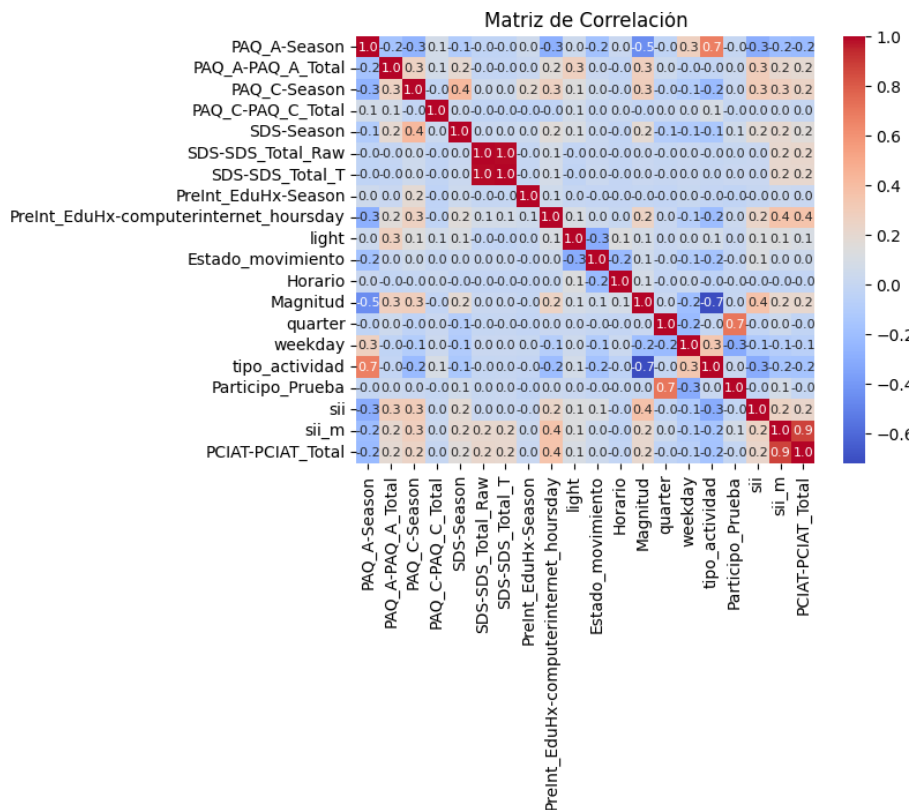
```
sns.heatmap(df[grupo_4].corr(), annot = True, cmap='coolwarm', fmt='.1f', annot_kws={"size": 6})
plt.title('Matriz de Correlación')
plt.show()
```



En este grupo que las preguntas 2,5 y 11 de cuestionario son las que tienen el coeficiente más alto alrededor de un 3.

```
grupo_5 = ["PAQ_A-Season", "PAQ_A-PAQ_A_Total", "PAQ_C-Season", "PAQ_C-PAQ_C_Total", "SDS-Season", "SDS-SDS_Total_Raw", "SDS-SDS_Total_T", "PreInt_EduHx-Season", "PreInt_EduHx-computerinternet_hoursday",
"light", "Estado_movimiento", "Horario", "Magnitud", "quarter", "weekday", "tipo_actividad", "Participo_Prueba", "sii", "sii_m", "PCIAT-PCIAT_Total"]

sns.heatmap(df[grupo_5].corr(), annot = True, cmap='coolwarm', fmt='.1f', annot_kws={"size": 8})
plt.title('Matriz de Correlación')
plt.show()
```



En este caso vemos que los días de la semana, tipo actividad, Magnitud son las que tienen el coeficiente más alto con relación a nuestras variables objetivo.

Estandarización variables

```
# Estamos seleccionando las variables predictoras y excluyendo la variable
x_standarizar = df.drop(columns=['sii_m', 'id', 'time_of_day', 'PAQ_A-PAQ_A_Total', 'PCIAT-PCIAT_Total'])
Y = df["sii_m"]
```

```
#Inicializar el standarizador
scaler = StandardScaler()
X_estandarizado = scaler.fit_transform(x_standarizar)
```

Importancias variables RandomForestRegressor

```
# Creando la instancia de RandomForestRegressor
rf = RandomForestClassifier()
```

```
# Se esta ajustando el modelo con los datos estandarizados
rf.fit(X = X_estandarizado, y = df["sii_m"])
```

```
RandomForestClassifier()
RandomForestClassifier()
```

```
rf.feature_importances_
```

```
array([0.00208339, 0.00599202, 0.00059544, 0.00228463, 0.00570386,  
       0.00223555, 0.00604985, 0.00654284, 0.006518 , 0.00263194,  
       0.00531989, 0.00588843, 0.00568579, 0.00152264, 0.00140333,  
       0.00165425, 0.00273674, 0.00226314, 0.00419119, 0.00084277,  
       0.00452121, 0.00082696, 0.0042936 , 0.00103194, 0.00429248,  
       0.0005892 , 0.00373279, 0.00067219, 0.00412941, 0.00064678,  
       0.00388503, 0.00058556, 0.00173591, 0.00173066, 0.00561955,  
       0.00425968, 0.01028886, 0.00575141, 0.00563654, 0.00469956,  
       0.00607221, 0.00468788, 0.00417838, 0.00115089, 0.00873738,  
       0.00728428, 0.00503874, 0.00742657, 0.00687446, 0.00188019,  
       0.00206668, 0.005397 , 0.00318673, 0.04112577, 0.05911827,  
       0.09414082, 0.01860656, 0.07760789, 0.02028864, 0.01031954,  
       0.02498324, 0.03029378, 0.02373986, 0.0270159 , 0.00499769,  
       0.03873148, 0.02460681, 0.05809711, 0.03791297, 0.05656918,  
       0.03788051, 0.02465801, 0.03391043, 0.00240266, 0.00623214,  
       0.00611866, 0.00219867, 0.00299606, 0.00261525, 0.00269405,  
       0.00241085, 0.00286142, 0.00291443, 0.00295065, 0.00263179,  
       0.00215001, 0.00301476, 0.00164423, 0.00065201, 0.00120094,  
       0.00221637, 0.00046976, 0.00053336, 0.00023109])
```

```
# Se esta creando un dataframe con los resultados |  
df_imp = pd.DataFrame({'nombre': columnas, "importancia": rf.feature_importances_ })  
df_imp
```

	nombre	importancia
0	Basic_Demos-Enroll_Season	0.002083
1	Basic_Demos-Age	0.005992
2	Basic_Demos-Sex	0.000595
3	CGAS-Season	0.002285
4	CGAS-CGAS_Score	0.005704
...	...	...
89	sii_m	0.001201
90	Magnitud	0.002216
91	tipo_actividad	0.000470
92	Estado_movimiento	0.000533
93	Horario	0.000231

94 rows × 2 columns

```
# Se estan ordenando los valores a traves de la columna importancia.  
df_imp = df_imp.sort_values("importancia", ascending = False)
```

```
# Se esta creando una columna que contenga el acumulando de varianza explicada.  
df_imp["imp_acu"] = df_imp["importancia"].cumsum()
```

```
# Se esta filtrando el dataframe por la columna imp_cum para saber cuantas y cuales variables se requieren para explicar el 90% de la varianza.  
df_imp[df_imp["imp_acu"] < 0.90]
```

	nombre	importancia	imp_acu
55	PCIAT-PCIAT_03	0.094141	0.094141
57	PCIAT-PCIAT_05	0.077608	0.171749
54	PCIAT-PCIAT_02	0.059118	0.230867
67	PCIAT-PCIAT_15	0.058097	0.288964
69	PCIAT-PCIAT_17	0.056569	0.345533
53	PCIAT-PCIAT_01	0.041126	0.386659
65	PCIAT-PCIAT_13	0.038731	0.425391
68	PCIAT-PCIAT_16	0.037913	0.463303
70	PCIAT-PCIAT_18	0.037881	0.501184
72	PCIAT-PCIAT_20	0.033910	0.535094
61	PCIAT-PCIAT_09	0.030294	0.565388
63	PCIAT-PCIAT_11	0.027016	0.592404
60	PCIAT-PCIAT_08	0.024983	0.617387
71	PCIAT-PCIAT_19	0.024658	0.642045
66	PCIAT-PCIAT_14	0.024607	0.666652
62	PCIAT-PCIAT_10	0.023740	0.690392
58	PCIAT-PCIAT_06	0.020289	0.710681
56	PCIAT-PCIAT_04	0.018607	0.729287
59	PCIAT-PCIAT_07	0.010320	0.739607
36	BIA-BIA_BMR	0.010289	0.749896
44	BIA-BIA_ICW	0.008737	0.758633
47	BIA-BIA_SMM	0.007427	0.766060
45	BIA-BIA_LDM	0.007284	0.773344
48	BIA-BIA_TBW	0.006874	0.780218
7	Physical-Height	0.006543	0.786761
8	Physical-Weight	0.006518	0.793279
74	SDS-SDS_Total_Raw	0.006232	0.799511
75	SDS-SDS_Total_T	0.006119	0.805630
40	BIA-BIA_FFMI	0.006072	0.811702
6	Physical-BMI	0.006050	0.817752
1	Basic_Demos-Age	0.005992	0.823744
11	Physical-HeartRate	0.005888	0.829632
37	BIA-BIA_DEE	0.005751	0.835384
4	CGAS-CGAS_Score	0.005704	0.841088
12	Physical-Systolic_BP	0.005686	0.846774
38	BIA-BIA_ECW	0.005637	0.852410
34	BIA-BIA_BMC	0.005620	0.858030
51	PAQ_C-PAQ_C_Total	0.005397	0.863427
10	Physical-Diastolic_BP	0.005320	0.868747
46	BIA-BIA_LST	0.005039	0.873785
64	PCIAT-PCIAT_12	0.004998	0.878783
39	BIA-BIA_FFM	0.004700	0.883482
41	BIA-BIA_FMI	0.004688	0.888170
20	FGC-FGC_GSND	0.004521	0.892692
22	FGC-FGC_GSD	0.004294	0.896985

```
# Este calculo nos permite saber cuantas variables se requiere para explicar el 90% de la varianza.
len(df_imp[df_imp["imp_acu"] < 0.90])
```

45

Se realizó un análisis de importancias de las variables utilizar el algoritmo RandomForestClassifier() que Indica cuánto contribuye cada característica al modelo para realizar predicciones precisas. Esto nos permitirá saber cuál de las variables o característica es más significativa para el modelo, ósea, para la variable sii_m. Aquí tenemos algunas observaciones:

- 46 variables explican el 90% de la varianza del modelo o comportamiento de la variable sii.
- La variable con mayor impacto la variable PCIAT-PCIAT_02.
- El 80% de la variación se explica solo con 28 variables.

PCA

```
# Crea una instancia de PCA con el parametro n_components = 0.95 lo que indica seleccionaremos el numero de componentes necesarios para explicar el 95% de la varianza de los datos.
pca = PCA(n_components = 0.95)

# Con este metodo de se calcula el numero de componentes y se transforman los datos originales a la nueva representacion de los datos con menos dimensiones.
x_pca = pca.fit_transform(X_estandarizado)
```

```
# Se crea un Dataframe con los datos con su representacion.
df_pca = pd.DataFrame(x_pca)
```

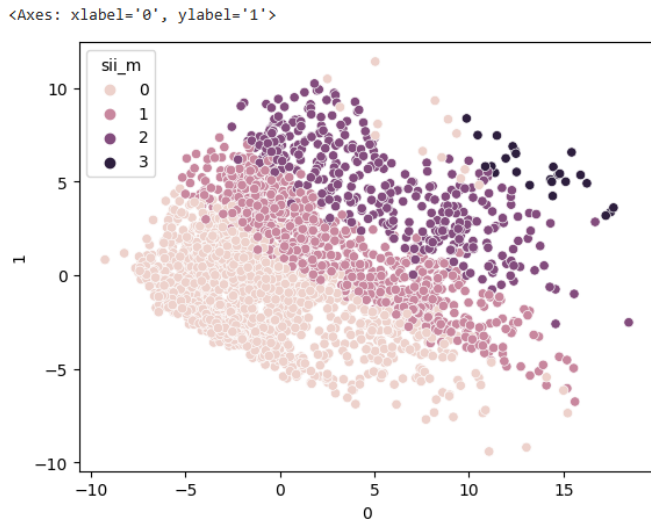
```
# Con este metodo estamos viendo la explicacion de la varianza por cada uno de los componentes resultante de aplicar el ajuste transformacion a los datos.
pca.explained_variance_ratio_.round(3)
```

```
array([0.26 , 0.093, 0.055, 0.035, 0.034, 0.029, 0.026, 0.022, 0.021,
       0.02 , 0.019, 0.019, 0.016, 0.015, 0.014, 0.014, 0.012, 0.012,
       0.011, 0.01 , 0.01 , 0.01 , 0.01 , 0.009, 0.009, 0.009, 0.008,
       0.008, 0.007, 0.007, 0.007, 0.007, 0.007, 0.006, 0.006, 0.006,
       0.006, 0.006, 0.005, 0.005, 0.005, 0.005, 0.005, 0.004,
       0.004, 0.004, 0.004, 0.004, 0.004, 0.004, 0.003, 0.003,
       0.003, 0.003, 0.003, 0.003])
```

```
# Con este metodo veremos la suma acumulativa de la varianza explicada por cada componente.
np.cumsum(pca.explained_variance_ratio_).round(3)
```

```
array([0.26 , 0.353, 0.407, 0.442, 0.476, 0.504, 0.531, 0.552, 0.573,
       0.592, 0.611, 0.63 , 0.646, 0.662, 0.676, 0.69 , 0.702, 0.713,
       0.724, 0.734, 0.745, 0.755, 0.764, 0.774, 0.782, 0.791, 0.799,
       0.807, 0.815, 0.822, 0.829, 0.836, 0.842, 0.848, 0.854, 0.86 ,
       0.866, 0.872, 0.877, 0.883, 0.887, 0.892, 0.897, 0.902, 0.906,
       0.91 , 0.915, 0.919, 0.923, 0.926, 0.93 , 0.933, 0.937, 0.94 ,
       0.943, 0.946, 0.949, 0.952])
```

```
sns.scatterplot(x = df_pca[0], y = df_pca[1], hue = df["sii_m"])
```



En esta grafica vemos un patrón claro de las cuatro categorías de la variable objetivo. Estos dos variables o componentes son los que explican por separado a mayor cantidad de varianza de los datos. Aquí podemos ver como el grupo numero 0 centra en el rango de -6 y 5 del componente 0. Otro patrón claro el grupo que tiene un rango de -4 hasta 9. De esa manera se ve el mismo comportamiento en los demás grupos.

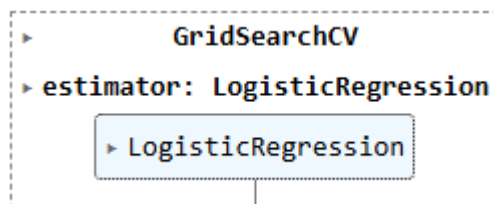
Modelados datos

División del conjunto de datos en pruebas y entrenamiento

```
X_train, X_test, Y_train, Y_test = train_test_split(x_pca, Y, test_size=0.3, stratify=Y, random_state=42)
```

Modelo de regresión logística

```
modelo = LogisticRegression(max_iter=1000, multi_class='ovr')
parametros = {'C': [0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09],
               "class_weight": ['balanced', None]}
model_GS = GridSearchCV(modelo, param_grid=parametros,
                        cv = 5, scoring='f1_macro')
model_GS.fit(X_train, Y_train)
```



```
print(model_GS.best_params_, "\nF1_marco: {}".format(round(model_GS.best_score_, 2)))
```

```
{'C': 0.09, 'class_weight': 'balanced'}
F1_marco: 0.67
```

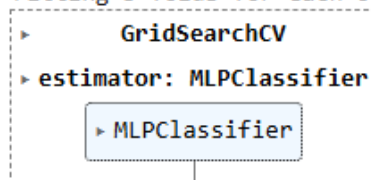
Como se puede visualizar estos son los mejores parámetros del modelo C': 0.09, 'class_weight': 'balanced' y su puntuación de f1_macro 0.67 que nos indica que tiene un rendimiento aceptable, pero con margen de mejora. Esto se debe puede deberse a que el modelo no está capturando todas las relaciones, lo intentaremos con otros más robustos.

Modelo de redes neuronales

```
modelo = MLPClassifier()
parametros = {'solver': ['lbfgs'],
              'max_iter': [100,200,300,500,700,800,1000], # Iteraciones máximas en cada red
              'alpha': 10.0 ** -np.arange(1, 10), # Parámetro de regularización L2 para evitar sobreajuste
              'hidden_layer_sizes':[(100,), (50,50), (64,32)], # Número de neuronas en cada capa
              'random_state':[0]}
```

```
model_RN = GridSearchCV(modelo, param_grid=parametros, cv = 3,
                        scoring='f1_macro', n_jobs=-1, verbose=10)
model_RN.fit(X_train, Y_train)
```

Fitting 3 folds for each of 189 candidates, totalling 567 fits



```
print(model_RN.best_params_, "\nf1_macro: {}".format(round(model_RN.best_score_,2)))
```

```
{'alpha': 0.1, 'hidden_layer_sizes': (50, 50), 'max_iter': 200, 'random_state': 0, 'solver': 'lbfgs'}
f1_macro: 0.85
```

Como podemos ver el valor arrojado por la métrica f1_micro fue 0.85 mayor que el modelo de regresión logística. Esto también nos dice que este modelo de red neuronal capturo relaciones más profundas entre las variables. El modelo tiene dos capas profundas la primera de 50 neuronas y la segunda de 50 con una iteración 200 epoc. También se puede ver el parámetro de regularización L2 es 0.1 es moderado, esto nos ayuda con el sobreajuste del modelo con los datos de entrenamiento.

Modelo XGBoots XGBClassifier

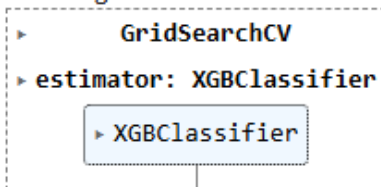
```
# Definir el modelo base
xgb = XGBClassifier(
    objective='multi:softprob', # multiclase con probabilidades
    num_class=4,
    use_label_encoder=False,
    eval_metric='mlogloss'      # log-loss multiclase
)
```

```
# Definir espacio de búsqueda de hiperparámetros
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [3, 5],
    'learning_rate': [0.01, 0.1],
    'subsample': [0.8, 1.0]
}
```

```
# Ejecutar GridSearchCV
grid = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid,
    scoring='f1_macro',
    cv=3,          # validación cruzada
    verbose=1,
    n_jobs=-1      # usa todos los núcleos disponibles
)

grid.fit(X_train, Y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits



```
print(grid.best_params_, "\nf1_macro: {}".format(round(grid.best_score_,2)))
```

Dado estos resultados, vemos que el modelo de Random forest tiene la mejor puntuación de la métrica f1_macro con 0.90, a pesar de que no por mucha diferencia, ya que el XGBClassifier arroja un 0.89. Esto nos indica que de los cuatro modelos analizados es mejor para prediciendo las clasificaciones. También nos dice que tiene una buena puntuación con las métricas precisión y recall, como sabemos la primera castiga muchos los falsos positivos y la otra los falsos negativos.

Dada esta puntuación el modelo que seleccionaremos el modelo de RandomForestClassifier.

Evaluación y Validación: RandomForestClassifier

```
# Crearemos otro objeto con los mejores parámetros
Op_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=9,
    max_features=10,
    class_weight='balanced',      # o 'balanced_subsample'
    random_state=42
)
```

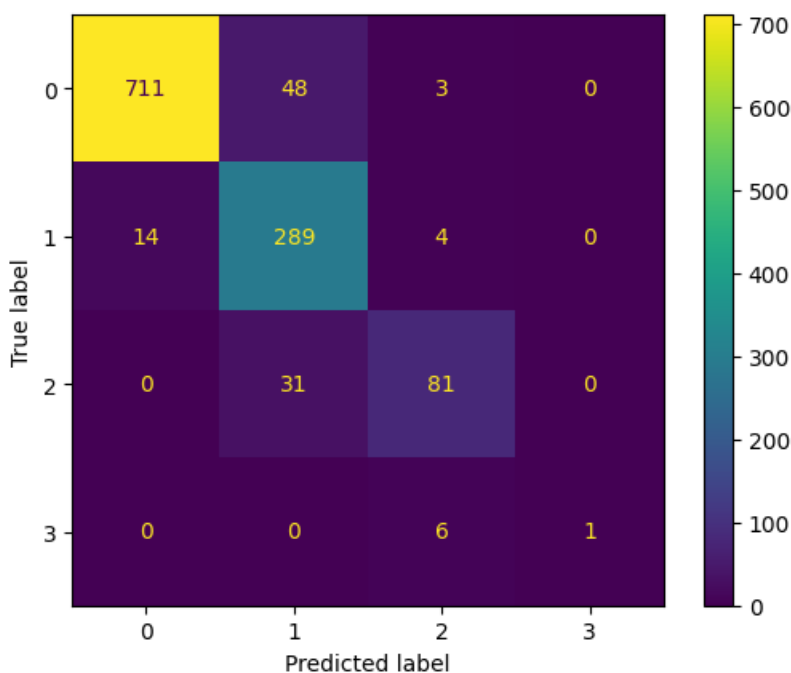
```
# Entrenamiento del modelo
Op_model.fit(X_train, Y_train)
```

```
RandomForestClassifier
RandomForestClassifier(class_weight='balanced', max_depth=9, max_features=10,
                        n_estimators=200, random_state=42)
```

```
y_predict = Op_model.predict(X_test)
y_predict
```

```
array([0, 1, 1, ..., 1, 1, 0])
```

```
cm = confusion_matrix(Y_test, y_predict)
ConfusionMatrixDisplay(confusion_matrix=cm).plot();
```



```
accuracy = accuracy_score(Y_test, y_predict)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9107744107744108
```

Como se puede ver la exactitud del modelo fue de 91%, mide la proporción de observaciones clasificadas correctamente de entre todas las observaciones, lo que significa que el 91% de las observaciones se clasificaron correctamente.

```
precision = precision_score(Y_test, y_predict, average = 'weighted')
print("Precision:", precision)
```

```
Precision: 0.919100390770121
```

Como podemos observar la métrica precision arrojo un valor de 91%, la cual mide la proporción de verdaderos positivos (observaciones positivas correctamente clasificadas) de todas las observaciones clasificadas como positivas, lo que significa que el 91% las observaciones positivas se clasificaron correctamente.

```
recall = recall_score(Y_test, y_predict, average = 'weighted')
print("Recall:", recall)
```

Como podemos observar la métrica Recall, que mide la proporción de verdaderos positivos de todos los casos positivos reales. En este caso, el modelo tiene un recuerdo de 0,91 lo cual significa que sólo clasificó correctamente el 91% de los casos positivos reales.

```
print(classification_report(y_predict, Y_test))
```

	precision	recall	f1-score	support
0	0.93	0.98	0.96	725
1	0.94	0.79	0.86	368
2	0.72	0.86	0.79	94
3	0.14	1.00	0.25	1
accuracy			0.91	1188
macro avg	0.69	0.91	0.71	1188
weighted avg	0.92	0.91	0.91	1188

A pesar de la buenas métricas que tiene el modelo en precision, recall y f1_score, vemos que la clase 3, que corresponden a personas adictivas no tiene muestras para predecir su clasificación. Esto se debe al desbalance que tiene la distribución de los valores de la columna `sii_m`, la cual es nuestra variable objetivo. Se intentó balancear con el parámetro `class_weight='balanced'` cuando creamos la instancia, sin embargo, no tuvo el efecto deseado.

Dada esta situación tenemos que la opción del modelo `XGBClassifier` con `f1_macro` 0.90 que es un balance entre la métrica recall y precision y nos dice que tiene un buen rendimiento de manera global.

Evaluación y Validación: XGBoostClassifier

```
# Se estan creando datos sintenticos con el metodo SMOTE.
# Estamos haciendo esto para balance los valores de la columna objetivo que tiene muy poco valores en la columna support cuando aplicamos las metrcias.
# El metodo, SMOTE nos ayudara a interpolar nuevos puntos entre cada muestra minoritaria y sus vecinos más cercanos.
```

```
smote = SMOTE(
    sampling_strategy={3: 80}, # Genera hasta 50 ejemplos sintéticos de la clase 3
    k_neighbors=2, # Vecinos para interpolar (elige < soporte real)
    random_state=42
)
X_res, Y_res = smote.fit_resample(X_train, Y_train)
```

```
xgb_model = XGBClassifier(
    objective='multi:softprob', # Para clasificación multiclase con probabilidades
    eval_metric='mlogloss', # Métrica estándar para multiclase (también puedes probar 'mlogloss' y 'merror')
    n_estimators=100, # Confirmado por GridCV
    max_depth=3, # Confirmado por GridCV
    subsample=0.8, # Confirmado por GridCV
    colsample_bytree=0.8, # Añadido: Mejora generalización reduciendo correlación entre árboles
    learning_rate=0.1, # Confirmado por GridCV
    gamma=0.1, # Regularización mínima por ahora (puedes tunearlo luego)
    reg_alpha=0.1, # L1 regularization (previene overfitting)
    reg_lambda=1.0, # L2 regularization (estabiliza el modelo)
    use_label_encoder=False, # Para evitar warnings en versiones modernas de XGBoost
    verbosity=1, # Muestra advertencias y errores mínimos
    random_state=42, # Reproducibilidad
    n_jobs=-1, # Uso completo de núcleos para paralelismo
)
```

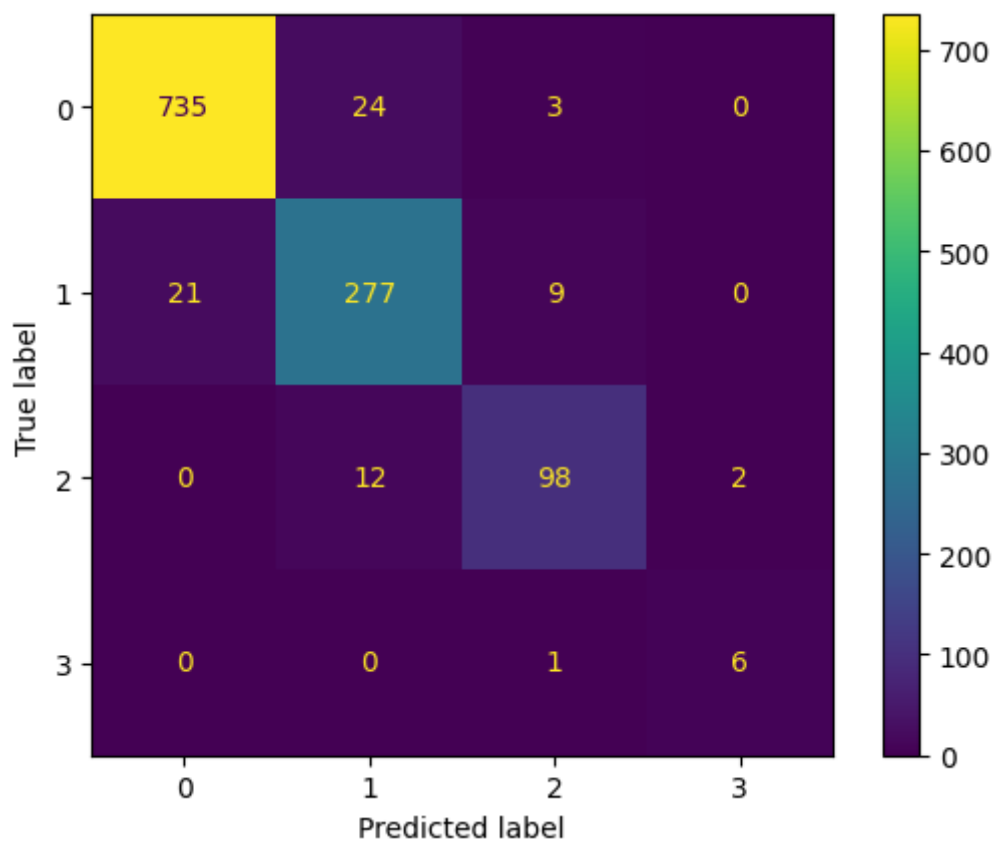
```
xgb_model.fit(X_res, Y_res)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytreet=0.8, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='mlogloss',
               feature_types=None, gamma=0.1, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.1, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=3,
               max_leaves=None, min_child_weight=None, missing=nan,
```

```
y_predict1 = xgb_model.predict(X_test)
y_predict1
```

```
array([0, 1, 1, ..., 1, 2, 0])
```

```
cm1 = confusion_matrix(Y_test, y_predict1)
ConfusionMatrixDisplay(confusion_matrix=cm1).plot();
```



```
accuracy = accuracy_score(Y_test, y_predict1)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9393939393939394
```

Como se puede ver la exactitud del modelo fue de 94%, mide la proporción de observaciones clasificadas correctamente de entre todas las observaciones, lo que significa que el 94% de las observaciones se clasificaron correctamente.

```
recall = recall_score(Y_test, y_predict1, average = 'weighted')
print("Recall:", recall)
```

Recall: 0.9393939393939394

Como podemos observar la métrica Recall, que mide la proporción de verdaderos positivos de todos los casos positivos reales. En este caso, el modelo tiene un recuerdo de 94%, lo que significa que sólo clasificó correctamente el 94% de los casos positivos reales.

```
print(classification_report(y_predict1, Y_test))
```

	precision	recall	f1-score	support
0	0.96	0.97	0.97	756
1	0.90	0.88	0.89	313
2	0.88	0.88	0.88	111
3	0.86	0.75	0.80	8
accuracy			0.94	1188
macro avg	0.90	0.87	0.89	1188
weighted avg	0.94	0.94	0.94	1188

Tras ir probando varios valores del parámetro `sampling_strategy`, correspondiente al método SMOTE, obtuvimos muy buenos resultados al fijar la cantidad de datos sintéticos en 80.

Las métricas `precision`, `recall` y `f1_score` mejoraron significativamente. También podemos ver que a nivel general la precisión muy buena, de cada clase predicha, en promedio, de cada 10 predicciones, 9 será correctas. En cuanto a la parte de `recall`, que se encarga de como este modelo identifica los casos reales, a nivel general y en promedio, de cada 10 casos reales solamente se le escapa 1.

En el caso de nosotros es de suma importancia que las predicciones sean confiables y identifique o reconozca los casos reales.

```

n_clases = 4

# Paso 1: Binarizar las etiquetas verdaderas
Y_test_bin = label_binarize(Y_test, classes=[0, 1, 2, 3])
# Y_test_bin tendrá forma [n_muestras, 4]

# Paso 2: Obtener las probabilidades por clase
y_t_pred_pro = xgb_model.predict_proba(X_test)
# y_t_pred_pro debe tener forma [n_muestras, 4]

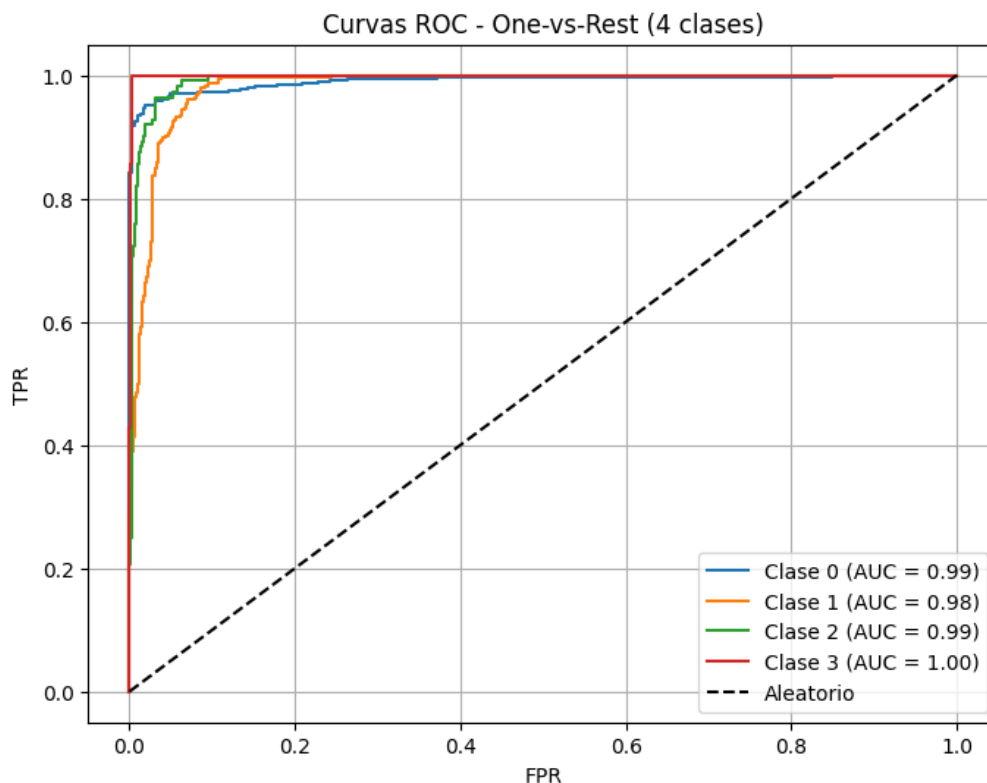
# Inicializar diccionarios para guardar los resultados
fpr = dict()
tpr = dict()
roc_auc = dict()

# Paso 3: Para cada clase, calcula la curva ROC (OvR)
for i in range(n_clases):
    y_true_bin = Y_test_bin[:, i]          # 1 si pertenece a la clase i, 0 si no
    y_score = y_t_pred_pro[:, i]           # Probabilidad asignada a la clase i
    fpr[i], tpr[i], _ = roc_curve(y_true_bin, y_score)
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure(figsize=(8, 6))
for i in range(n_clases):
    plt.plot(fpr[i], tpr[i], label=f'Clase {i} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('Curvas ROC - One-vs-Rest (4 clases)')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

```



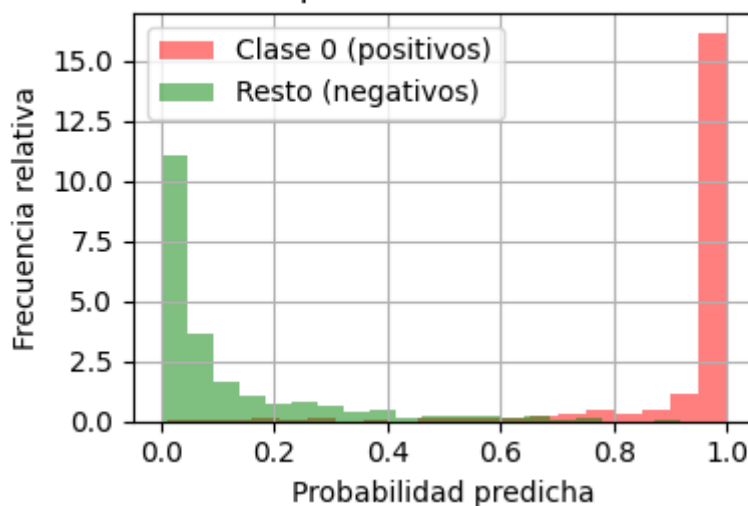
Estos gráficos son representan la curva ROC los cuales se utilizar para caso binarios y saber que tanto bien clasifica un modelo binario. En este caso aplicamos para un modelo multiclase utilizando la estrategia OvR. Este método consiste en comparar una clase con el resto, la clase elegida toma el valor de tasa de verdaderos positivos y el resto de tasa de falsos positivos. Quise utilizar esta estrategia ya que quería ver, de manera

gráfica, que también discriminaba las clases, o para ser más específico que también clasificaba o diferencia entre las clases.

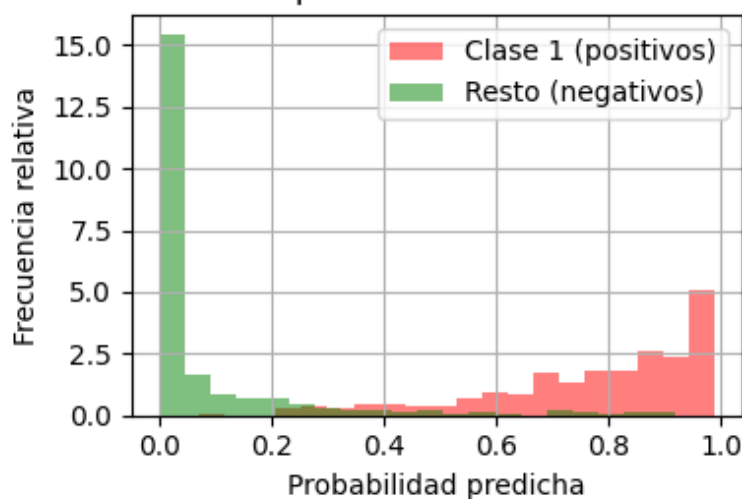
A nivel general vemos que el modelo tiene una muy precisión al momento de diferenciar y clasificar entre las clases. Vemos a modos de ejemplo, que el modelo, en nuestro caso, diferencia muy aquellos niños y adolescente tiene uso problemático del internet y aquellos que no. De igual manera que clasifica muy aquellos que tienen una conducta media o moderada, los cuales pueden corregirse antes de que asuman un estatus de severo o su salud se vea afectada.

```
representar_histogramas_ovr(Y_test, y_t_pred_pro, figsize=(4, 3))
```

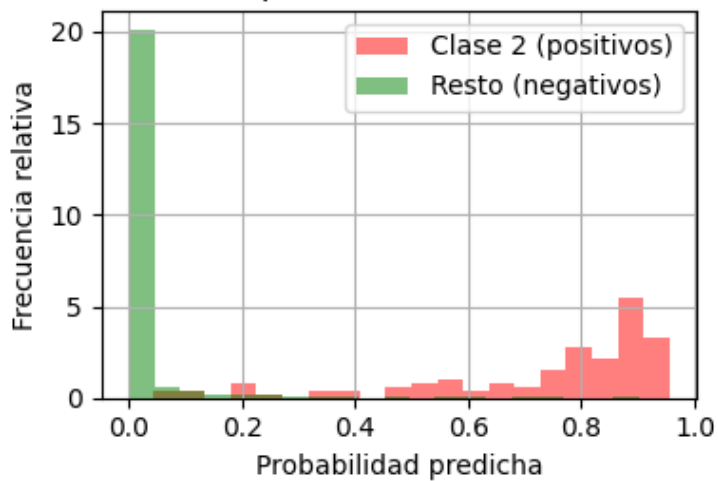
Distribución de probabilidades - Clase 0 vs Resto



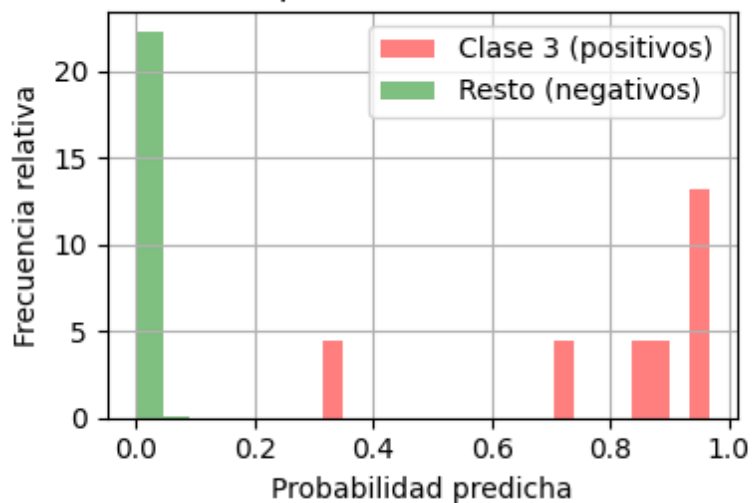
Distribución de probabilidades - Clase 1 vs Resto



Distribución de probabilidades - Clase 2 vs Resto



Distribución de probabilidades - Clase 3 vs Resto



En estos histogramas podemos seguir confirmando la calidad del modelo a la hora de realizar buenas clasificaciones o discriminaciones entre clases. A nivel general vemos que el modelo separa muy bien las clases. En el caso de la clase 3, en nuestro caso de estudio, niños o adolescente que tiene una conducta de estatus grave con el uso problemático del internet, lo discrimina muy bien. Esto es bien importante para este caso, ya que es crucial de predecir una clasificación de este tipo.

Conclusiones

- Es posible desarrollar un modelo predictivo confiable y con precisión para identificar el uso problemático del internet de niños y adolescentes a partir de condiciones y actividades físicas.
- El comportamiento adictivo es más probable en participantes de sexo masculino que femenino.
- El 38% de los participantes podrían tener un uso problemático del internet.
- Las personas con obesidad o sobrepeso mostraron más propensión a tener uso problemático del internet.
- El modelo XGBOOSTClassifier fue el mas robusto y confiable de todos lo entrenado, el que tuvo mejor rendimiento y pudo discriminar entre las clases.
- El descuido de las tareas del hogar es un fuerte indicador de una persona con uso problemático del internet.
- La desobediencia tiene una alta correlación con el uso problemático del internet.
- El rendimiento académico si es afectado por el uso problemático del internet.
- El nivel de agua corporal es un buen indicador del uso problemático del internet.
- El gasto energético diario es otro buen indicador del uso problemático del internet.
- La edad está muy relacionada con el uso problemático del internet.

Referencia bibliográfica

<https://www.infocop.es/alcohol-tabaco-uso-problematico-de-internet-en-adolescentesoeda/>

<https://onlinelibrary.wiley.com/doi/10.1155/2015/291743>

<https://reviberopsicologia.iberu.edu.co/article/view/rip.11309>

<https://www.sciencedirect.com/science/article/abs/pii/S0747563216304198?via%3Dihb>

<https://www.sciencedirect.com/science/article/abs/pii/S0747563215302041?via%3Dihb>

https://scielo.isciii.es/scielo.php?script=sci_arttext&pid=S1137-66272007000200004

https://irep.ntu.ac.uk/id/eprint/31776/1/PubSub9230_Griffiths.pdf

https://www.researchgate.net/publication/269985069_Internet_Addiction_Disorder_and_Internet_Gaming_Disorder_are_Not_the_Same

<https://youtu.be/qalCaPIbWaE?si=EF1PsBpH70QywmorR>

https://youtu.be/WuSRQEw8Gow?si=Rtsu_ncH8Tgfp4k5

<https://youtu.be/m3jVJco34D0?si=l2eAvncovdBaBh-l>